

Program Extraction and Specification Extraction: A Sound Shallow Translation of Dependent Type Theory into First-Order Logic

AI draft of July 7, 2026

Abstract

Hammer-style automation for proof assistants based on the Calculus of Inductive Constructions (CIC) translates goals and premises into untyped first-order logic (FOL) and discharges the resulting problems with automated theorem provers. We give a precise formulation of a translation architecture that factors this translation through the intermediate representation of Letouzey’s program extraction: the translation of a definition $c := t : \tau$ consists of (i) *program extraction* — the erasure $\mathcal{E}(t)$ of the body into an untyped functional language, compiled to *per-constructor pattern equations* in FOL, and (ii) *specification extraction* — a realizability reading of the type τ as a first-order formula about the FOL constant $\hat{c}$. Both components are *shallow*: propositions and specifications are mapped directly to first-order connectives and quantifiers computed at translation time, refinement-type payloads are expanded inline at every occurrence, and no inhabitation predicate mediates the logical structure. We define a monomorphic first-order fragment CIC^- of CIC with inductive datatypes, refinement types, inductive predicates, singleton eliminations and structural fixpoints, and prove the translation sound: from any well-formed definitional environment we construct a first-order model of all emitted axioms — over the open term model of the erasure calculus, with explicit ill-typed “junk” elements — in which the translation of the goal reflects a proof-irrelevant, classical truth semantics of the source. Consequently the translation of a consistent environment is consistent, and no refutable goal becomes provable. As a by-product we obtain a rigorous account of a known design question for such translations: the classical single *guarded disjunctive* axiom for a pattern match is false at junk elements — its type guard is mandatory — whereas the per-constructor equations produced by extraction hold at *all* elements of the model, so the guards on matched terms can be dropped, and the two forms are interderivable relative to the structural axioms of the datatype.

Contents

1	Introduction	3
1.1	The architecture	4
1.2	Contributions	5
1.3	What soundness means here	7
1.4	Relation to the implemented translation	7
2	The source calculus CIC^-	8
2.1	Syntax	8
2.2	Environments	10
2.3	Typing	10
2.4	Reduction and conversion	11
2.5	The shallow fragment	13
2.6	Relation to CIC	13
3	The intermediate language and program extraction	14
3.1	The calculus $\lambda_{\square}$	15
3.2	Erasure	16
4	The translation	17
4.1	Signature	18
4.2	Compilation of erased programs	18
4.3	Specification extraction	21
4.4	The theory $\Delta(\mathbb{E}, G)$	23
5	Semantics	24
5.1	The domain	24
5.2	Interpretation of types, propositions and predicates	24
5.3	The first-order structure $\mathfrak{M}$	27
5.4	Adequacy of the term translation and of compilation	27
5.5	Coincidence: the translation reflects the semantics	29
6	Soundness	30
6.1	Satisfaction of contexts and guard disciplines	30
6.2	The fundamental lemma	32
6.3	Validity of the emitted theory	35
6.4	Main theorem and corollaries	36
7	Per-constructor equations versus guarded disjunctive case axioms	37
7.1	Semantic status	38
7.2	Logical relationship	39
7.3	Clause shapes and prover mechanics	40

8	Shallowness	41
9	Related work	43
10	Limitations and future work	44
10.1	Polymorphism and monomorphisation	44
10.2	Indexed families and large elimination	45
10.3	Well-founded recursion: a hard constraint	45
10.4	Transport erasure and UIP	46
10.5	Function extensionality fails in the term model	46
10.6	Other presentational restrictions	46
10.7	Completeness	47
10.8	Proof-theoretic soundness and implementation	47
A	Confluence of $\lambda_{\square}$	51

1 Introduction

A *hammer* is a proof-automation tool that, given a goal stated in the logic of a proof assistant and the ambient library of previously established facts, selects a set of plausibly relevant premises, translates the goal and the premises into the input logic of external automated theorem provers (ATPs), runs the ATPs, and finally reconstructs any proof found inside the proof assistant [BKPU16]. For proof assistants based on dependent type theory, the reference tool is CoqHammer [CK18, CK26] for Rocq (formerly Coq), whose translation maps the Calculus of Inductive Constructions (CIC) into untyped first-order logic (FOL).

The efficiency of the whole enterprise hinges on the translation being *shallow* [Cza18]: a source proposition must become a first-order *formula*, with the source connectives and quantifiers mapped directly to their first-order counterparts, so that the ATP’s highly optimised handling of first-order primitives — clausification, resolution, superposition, term indexing — does real logical work on the translated problem. The alternative, a *deep* embedding in which propositions become first-order *terms* related by an inhabitation predicate $\top(x, \tau)$, forces the ATP to simulate the object logic one bridge-axiom instantiation at a time and to synthesise first-order terms standing for proof terms; this is well understood to be dramatically less effective (see the discussion in [Cza18, §1.1] and Section 8 below).

For the *propositional* skeleton of a statement, shallowness is achieved by the translation of [CK18]: universally quantified implications become guarded first-order formulas, equality becomes first-order equality, and so on. Two related parts of the translation, however, have remained essentially deep, and both concern exactly the places where dependent type theory diverges from first-order logic:

1. *Definitions by pattern matching.* The translation of [CK18] renders a match $\text{case}(x; \cdot; \mathbf{O} \Rightarrow t_1 \mid \mathbf{S}y \Rightarrow t_2)$, lifted to a fresh constant F , as a single *guarded disjunctive axiom*

$$\forall x. \text{nat}(x) \rightarrow (x \approx \mathbf{O} \wedge F(x) \approx t'_1) \vee (\exists y. \text{nat}(y) \wedge x \approx \mathbf{S}(y) \wedge F(x) \approx t'_2),$$

a formula that clausifies into many long non-Horn clauses with Skolem functions and that superposition provers cannot orient into rewrite rules.

2. *Types with propositional content.* A constant whose type mixes computation with specification — the running example, from the CoqHammer problem list, is

$$\begin{aligned} h : \forall x y z : \text{nat}. x = y \wedge y = z \rightarrow \{u : \text{nat} \mid x = u\} \\ h \ x \ y \ z \ p := \text{match } p \text{ with } \text{conj } p_1 \ p_2 \Rightarrow \\ \text{exist } (\lambda u. x = u) \ z \ (\text{eq_trans } p_1 \ p_2) \end{aligned}$$

— receives *no* usable axiom at all: the match on the proof p is translated to an opaque constant, and the type is translated to an opaque atom $\text{HasType}(h, \text{sig}(\text{nat}, P))$ that no other axiom connects to anything. The information is present but buried inside an inhabitation atom: a local relapse into the deep embedding.

Both phenomena have a common diagnosis, already conjectured in [CK18, §9]: the translation should be *factored through the intermediate representation of program extraction* in the sense of Letouzey [Let04, Let03, Let08]. This paper carries the factorisation out precisely and proves it sound.

1.1 The architecture

The slogan is:

$$\text{translation} = \text{program extraction} + \text{specification extraction}.$$

For a definition $c := t : \tau$ in the source environment, the translation emits two groups of first-order axioms.

Program extraction. The body t is erased, following Letouzey’s function $\mathcal{E}$, into a term of an untyped functional calculus $\lambda_\square$ (Section 3): proofs and propositional arguments disappear, matches on propositional singletons (conjunctions, equations, $\perp$) collapse to their unique branch, and refinement values exist $a \ p$ unbox to a . The erased program is then compiled — by lambda-lifting, case flattening and pattern-match compilation (Section 4) — into a set of *defining equations*, one per constructor pattern. For the

standard addition function the output is literally the recursive program, read as a first-order rewrite system:

$$\forall m. \widehat{\text{add}}(\text{O}, m) \approx m, \quad \forall y m. \widehat{\text{add}}(\text{S}(y), m) \approx \text{S}(\widehat{\text{add}}(y, m)).$$

These are orientable unit equations — demodulators for superposition provers, ideal trigger material for SMT solvers — in place of the guarded disjunction above. For h , erasure yields $\mathcal{E}(t) = z$ and hence the single equation $\forall x y z. \widehat{h}(x, y, z) \approx z$.

Specification extraction. The type τ is translated by a realizability reading $\mathcal{G}$ in the tradition of Kleene [Kle45] and Paulin-Mohring [PM89], defined structurally on types: informative quantification becomes guarded first-order quantification, propositional premises become implications with the (erased) proof argument dropped, and — the essential new clause — a refinement type $\{x:\sigma \mid \varphi\}$ is expanded *inline, at every occurrence*, into the conjunction of the carrier guard and the payload formula. For h this produces the specification axiom

$$\begin{aligned} \forall x y z. \text{nat}(x) \rightarrow \text{nat}(y) \rightarrow \text{nat}(z) \rightarrow \\ (x \approx y \wedge y \approx z \rightarrow \text{nat}(\widehat{h}(x, y, z)) \wedge x \approx \widehat{h}(x, y, z)), \end{aligned}$$

which is exactly the statement of h ’s type, as a first-order formula. No `HasType` atom, no `sig`, no `exist` and no projection appears anywhere in the output: the specification is shallow in precisely the sense of [Cza18, §1.1].

The two components are the two halves of extraction correctness: the equations describe the extracted program; the specification axiom is the statement that the program *realizes* its type, which is Letouzey’s simulation predicate $\llbracket \tau \rrbracket$ [Let04, §2.4] applied to the extracted constant.

1.2 Contributions

This paper makes the architecture precise and proves it correct. Specifically:

1. We define a source calculus CIC^- (Section 2): a monomorphic, first-order fragment of CIC with inductive datatypes, primitive refinement types, inductive predicates, propositional singleton eliminations into informative positions (conjunction, equality, absurdity), structural fix-points, and full first-order intuitionistic logic with induction at the proof level. The calculus is *stratified*: terms, proofs, types, propositions and predicates are separate syntactic categories, so the erasure zones of extraction are syntactic rather than derived from sort analysis. Section 2.6 explains how the fragment arises from full CIC by the instance-classification analysis implemented in CoqHammer.

2. We define the erasure $\mathcal{E}$ from CIC^- into an untyped calculus $\lambda_\square$ with constructors, case and fixpoints (Section 3), and prove the two syntactic pillars of program extraction in this setting: erasure commutes with substitution, and erasure maps source conversion to $\lambda_\square$ -conversion. $\lambda_\square$ is an orthogonal rewrite system; we prove its confluence in Appendix A by the Tait–Martin-Löf/Takahashi method.
3. We define the translation (Section 4): the compilation of erased programs to per-constructor pattern equations with case flattening and lambda-lifting; the specification extraction $\mathcal{G}$, the proposition translation $\mathcal{F}$ and the term translation $\mathcal{C}$; and the emitted theory $\Delta(\mathbb{E}, G)$ for an environment $\mathbb{E}$ and goal G . All three translation functions are structural — there is no normalisation requirement on the source — and produce formulas built exclusively from first-order connectives, per-datatype guard predicates, per-instance inductive-predicate atoms, and equality.
4. We prove soundness (Sections 5 and 6). From any well-formed definitional environment we construct a first-order structure $\mathfrak{M}$ whose domain is the set of closed $\lambda_\square$ terms modulo conversion — crucially including “junk” elements denoting no source value — and a proof-irrelevant classical truth semantics $\langle \cdot \rangle$ for source propositions, and show: every axiom of $\Delta(\mathbb{E}, G)$ holds in $\mathfrak{M}$ (Theorem 6.8), and $\mathfrak{M} \models \mathcal{F}(G)$ implies $\langle G \rangle = 1$ (Lemma 5.11). Hence if $\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \mathcal{F}(G)$ then G is true in the semantics; in particular no goal refutable in CIC^- becomes provable after translation, and $\Delta(\mathbb{E}, G)$ is consistent (Theorem 6.9 and Corollary 6.10). The technical core is a fundamental lemma (Lemma 6.6) proved by induction on typing derivations, with a rank-indexed treatment of structural fixpoints in the style of type-based termination [BFG⁺04, Gim95].
5. We settle, rigorously, the design question that motivated the factorisation (Section 7): the guarded disjunctive case axiom of [CK18, §5.2] is *false* in $\mathfrak{M}$ if its guard is omitted — we exhibit the junk countermodel — while the per-constructor equations hold at all elements of $\mathfrak{M}$, junk included, so they need no guards at all; and the two axiomatisations are interderivable in FOL relative to the structural axioms (inversion, discrimination, injectivity) that the translation emits anyway. The choice between them is therefore purely operational, and Section 7.3 quantifies the operational difference in clause shapes.
6. We articulate the shallowness of the translation as a checklist of verifiable properties and verify them clause by clause (Section 8), and we delimit precisely what is *not* covered — polymorphism, indexed inductive families, well-founded recursion, large elimination — together with the

known hard constraint for extending the architecture to well-founded recursion (Section 10).

1.3 What soundness means here

Following automated-reasoning terminology [Cza18, BBPS16], a hammer translation is *sound* when derivability of the translated goal from the emitted theory guarantees that the source goal holds. Concretely, in the form we prove (Theorem 6.9): if $\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \mathcal{F}(G)$ then G is true in a fixed semantics of the source development (Section 5). This is weaker than exhibiting a source *proof* of G — a distinction we return to below — but it already excludes the failure that matters: no goal that is false in that semantics, and in particular none refutable in the source ($\mathbb{E} \vdash \neg G$), has a derivable translation (Corollary 6.11). (The converse property, that provable goals remain provable, is *completeness*; hammers deliberately trade it away, and the present translation is complete only on a fragment. See Section 10.7.) Since the final arbiter in a hammer is proof reconstruction inside the proof assistant, soundness is not a safety requirement but a quality requirement — an unsound translation wastes ATP time on phantom proofs and poisons premise-selection learning [BKPU16] — and, more importantly for this paper, the soundness *proof* is the specification of the translation: it identifies exactly which axioms may be emitted, which guards are mandatory, and why.

We prove soundness model-theoretically, against a semantics that interprets source propositions classically and proof-irrelevantly (Section 5). Both choices are forced: the ATPs are classical, and the translation erases proofs, so it validates proof irrelevance by construction — the corresponding necessity was isolated for the core embedding in [Cza18, Example 52]. The semantics is nevertheless *standard* on the first-order data of interest: the interpretation of `nat` in our model is (isomorphic to) the natural numbers (Proposition 6.12), so for concrete arithmetic-style goals “true in the semantics” means true.

A proof-theoretic soundness theorem in the style of [Cza18] — reconstructing a source proof term from a first-order derivation — is strictly stronger and correspondingly harder in the presence of inductive types; we leave it as future work (Section 10), noting that the model-theoretic statement is the one with operational consequences for a hammer.

1.4 Relation to the implemented translation

The calculus CIC^- and the translation of this paper are an idealisation of a design for CoqHammer, not a description of its current code: the current tool emits the guarded disjunctive case axioms of Section 7 and translates the type of h to an opaque inhabitation atom. The idealisation is faithful in the

other direction: every axiom family emitted by the translation of this paper has a direct implementation site in the existing translation pipeline, and the fragment CIC^- is the image of CIC under analyses (proposition detection, singleton classification, refinement classification, monomorphisation) that are either implemented or specified for implementation. We indicate the correspondences where relevant but keep the development self-contained.

2 The source calculus CIC^-

This section defines the source calculus of the translation: a monomorphic, first-order fragment of the Calculus of Inductive Constructions. The calculus is presented in a *stratified* style: instead of a single syntactic category of terms classified by sorts, we keep separate categories for set types, propositions, predicates, terms and proofs. The stratification builds the outcome of CIC 's sort analysis — which subterms are computational, which are logical, which are type-level — into the grammar, so that the erasure function of Section 3 and the translation of Section 4 are definable by structural recursion on raw syntax, with no typing oracle. Section 2.6 discusses how the fragment is carved out of full CIC .

2.1 Syntax

We fix countably infinite, pairwise disjoint sets of *term variables* (written $x, y, z, w, f, \dots$), of *proof variables* ($p, h, \dots$) and of *predicate variables* ($q, \dots$), and disjoint sets of names for *datatypes* (I), for *datatype constructors* ($\mathbf{c}, \mathbf{d}$), for *inductive predicates* (J), for their *clause constructors* ($\mathbf{k}$), and for *constants* (c).

Definition 2.1 (Syntax of CIC^-). The syntactic categories of *set types* σ, τ , *propositions* φ, ψ , *predicate kinds* K , *predicate expressions* P , *terms*

t, a, b and *proofs* π are defined by the following grammar.

$$\begin{aligned}
\sigma, \tau &::= I \mid \Pi x:\sigma. \tau \mid \varphi \rightarrow \tau \mid \{x:\sigma \mid \varphi\} \\
\varphi, \psi &::= \top \mid \perp \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \forall x:\sigma. \varphi \mid \exists x:\sigma. \varphi \\
&\quad \mid \forall q:K. \varphi \mid a =_\sigma b \mid J(\vec{a}) \mid q(\vec{a}) \\
K &::= (\sigma_1, \dots, \sigma_n) \rightarrow \mathbf{Prop} \quad (n \geq 1) \\
P &::= q \mid \lambda(x_1:\sigma_1, \dots, x_n:\sigma_n). \varphi \\
t, a, b &::= x \mid c \mid t a \mid t \pi \mid \lambda x:\sigma. t \mid \lambda p:\varphi. t \mid c(\vec{a}) \\
&\quad \mid \mathbf{case}(a; z.\tau; \vec{br}) \mid \mathbf{fix} f:T. t \mid \mathbf{refine}(t, \pi) \mid \mathbf{val}(t) \\
&\quad \mid \mathbf{split}(\pi; p_1 p_2. t) \mid \mathbf{cast}(\pi; z.\tau; t) \mid \mathbf{efq}(\pi; \sigma) \\
\pi &::= p \mid c \mid \lambda p:\varphi. \pi \mid \pi \pi' \mid \lambda x:\sigma. \pi \mid \pi a \mid \Lambda q:K. \pi \mid \pi P \mid \langle \pi_1, \pi_2 \rangle \mid \pi.1 \mid \pi.2 \\
&\quad \mid \iota_1(\pi) \mid \iota_2(\pi) \mid \mathbf{vcase}(\pi; p_1.\pi_1; p_2.\pi_2) \mid \langle a, \pi \rangle_{\exists x:\sigma. \varphi} \mid \mathbf{ecase}(\pi; x p. \pi') \\
&\quad \mid \mathbf{tt} \mid \mathbf{efq}(\pi; \varphi) \mid \mathbf{refl}_\sigma(a) \mid \mathbf{rew}(\pi; z.\psi; \pi') \mid \mathbf{k}(\vec{a}; \vec{\pi}) \mid \mathbf{ind}_J(\vec{z}.\psi; \vec{\pi}; \pi) \\
&\quad \mid \mathbf{ind}_I(z.\psi; \vec{\pi}; a) \mid \mathbf{valid}(t)
\end{aligned}$$

where a branch list $\vec{br}$ has the form $\{c_1(\vec{y}_1) \Rightarrow t_1 \mid \dots \mid c_m(\vec{y}_m) \Rightarrow t_m\}$ with one branch for every constructor of the datatype concerned, and T in $\mathbf{fix} f:T. t$ ranges over set types. In $\mathbf{case}(a; z.\tau; \vec{br})$ the variable z binds into the motive τ ; in $\{x:\sigma \mid \varphi\}$ the variable x binds into φ ; the binding structure of the remaining forms is the evident one. Terms of the form $\mathbf{refine}(t, \pi)$ carry a refinement-type annotation $\{x:\sigma \mid \varphi\}$, and $\langle a, \pi \rangle$ carries its $\exists$ -type; we omit the annotations when they are irrelevant. We write $\neg\varphi$ for $\varphi \rightarrow \perp$ and $\varphi \leftrightarrow \psi$ for $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$. Terms, proofs, types, propositions and predicate expressions are identified up to α -equivalence, and substitution is capture-avoiding.

Three points deserve comment.

First, the calculus has *three* function spaces at the term level: $\Pi x:\sigma. \tau$ (informative argument), $\varphi \rightarrow \tau$ (propositional premise; note that the premise does not bind a variable into τ , so the type is non-dependent, although the term-level abstraction $\lambda p:\varphi. t$ does bind p in the body), and the proof-level implications and quantifications. Dependency of types on terms enters exclusively through refinement types $\{x:\sigma \mid \varphi\}$ and through the propositions embedded in types — there are no type-level functions, no universes and no large elimination, so set types are *rigid*: their outermost former is stable under conversion (Lemma 2.5).

Second, the singleton eliminations of CIC appear as the primitive term formers $\mathbf{split}(\pi; p_1 p_2. t)$ (elimination of a conjunction proof into a term), $\mathbf{cast}(\pi; z.\tau; t)$ (transport along an equality proof) and $\mathbf{efq}(\pi; \sigma)$ ($\perp$ -elimination into a term). These are exactly the propositional eliminations into informative positions that CIC's elimination restriction permits: at most one constructor, all arguments propositional [The26, Let03].

Third, refinement types come with the constructor $\text{refine}(t, \pi)$ (pairing a value with a proof of the payload), the projection $\text{val}(t)$ (the value) and the proof former $\text{valid}(t)$ (the payload); this is `sig/exist/proj1_sig/proj2_sig` after Letouzey’s informative-singleton unboxing [Let04, §4.3] has been applied at the level of the calculus itself.

2.2 Environments

Definition 2.2 (Declarations and environments). A *global environment* $\mathbb{E}$ is a finite list of declarations of the following three kinds.

1. A *datatype declaration* $\text{data } I := \{c_1 : (\vec{\sigma}_1); \dots; c_m : (\vec{\sigma}_m)\}$ with $m \geq 1$ constructors. Each constructor argument type σ_{ij} is either the datatype name I itself (a *recursive argument*) or a closed set type over the preceding environment in which I does not occur.
2. An *inductive-predicate declaration* $\text{pred } J : (\vec{\sigma}) \rightarrow \text{Prop}$ with a finite list of clauses

$$k_j : \forall \vec{x}_j : \vec{\tau}_j. \varphi_{j1} \rightarrow \dots \rightarrow \varphi_{j\ell_j} \rightarrow J(\vec{u}_j)$$

where the index types $\vec{\sigma}$ are closed set types over the preceding environment, the $\vec{\tau}_j$ are set types over the preceding environment (possibly mentioning $\vec{x}_j$ bound earlier in the same telescope, through refinements), the indices $\vec{u}_j$ are terms over $\vec{x}_j$, and each premise φ_{ji} is either of the form $J(\vec{v})$ (a *recursive premise*) or a proposition over the preceding environment and $\vec{x}_j$ in which J does not occur.

3. A *definition*, either a *term definition* $c := t : \sigma$ with σ a closed set type and t a closed term, or a *lemma* $c := \pi : \varphi$ with φ a closed proposition and π a closed proof.

All names declared in $\mathbb{E}$ are distinct. An environment consisting only of declarations of kinds (1)–(3) — i.e., in which every constant has a body — is called *definitional*; this paper considers definitional environments only (see Remark 6.13 for postulated axioms).

Definition 2.3 (Contexts). A *context* Γ is a finite list of declarations $x : \sigma$, $p : \varphi$ and $q : K$ with pairwise distinct variables, each type, proposition or kind well formed over the preceding declarations (Figure 1).

2.3 Typing

The judgements of the calculus are: $\vdash \mathbb{E} \text{ wf}$ (environment well-formedness); and, relative to a fixed well-formed $\mathbb{E}$ which we leave implicit, $\Gamma \vdash \sigma \text{ type}$, $\Gamma \vdash \varphi \text{ prop}$, $\Gamma \vdash P : K$, $\Gamma \vdash t : \sigma$, and $\Gamma \vdash \pi : \varphi$. The rules are given in Figures 1 to 3. Environment well-formedness requires each declaration to be well formed over its prefix: the type and proposition components of

$$\begin{array}{c}
\frac{I \in \mathbb{E}}{\Gamma \vdash I \text{ type}} \quad \frac{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \tau \text{ type}}{\Gamma \vdash \Pi x:\sigma. \tau \text{ type}} \quad \frac{\Gamma \vdash \varphi \text{ prop} \quad \Gamma \vdash \tau \text{ type}}{\Gamma \vdash \varphi \rightarrow \tau \text{ type}} \\
\frac{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \varphi \text{ prop}}{\Gamma \vdash \{x:\sigma \mid \varphi\} \text{ type}} \quad \frac{}{\Gamma \vdash \top \text{ prop}} \quad \frac{}{\Gamma \vdash \perp \text{ prop}} \\
\frac{\Gamma \vdash \varphi \text{ prop} \quad \Gamma \vdash \psi \text{ prop}}{\Gamma \vdash \varphi \otimes \psi \text{ prop}} \quad \frac{}{\Gamma \vdash \sigma \text{ type} \quad \Gamma, x:\sigma \vdash \varphi \text{ prop}} \quad \frac{}{\Gamma \vdash Qx:\sigma. \varphi \text{ prop}} \quad \frac{}{\Gamma \vdash a : \sigma \quad \Gamma \vdash b : \sigma} \\
\frac{}{\Gamma \vdash \forall q:K. \varphi \text{ prop}} \quad \frac{}{\Gamma \vdash a =_{\sigma} b \text{ prop}} \\
\frac{J : (\vec{\sigma}) \rightarrow \text{Prop} \in \mathbb{E} \quad \Gamma \vdash \vec{a} : \vec{\sigma}}{\Gamma \vdash J(\vec{a}) \text{ prop}} \quad \frac{q : (\vec{\sigma}) \rightarrow \text{Prop} \in \Gamma \quad \Gamma \vdash \vec{a} : \vec{\sigma}}{\Gamma \vdash q(\vec{a}) \text{ prop}} \\
\frac{\Gamma \vdash \sigma_i \text{ type} \ (1 \leq i \leq n)}{\Gamma \vdash (\sigma_1, \dots, \sigma_n) \rightarrow \text{Prop kind}} \quad \frac{q : K \in \Gamma}{\Gamma \vdash q : K} \\
\frac{\Gamma, x_1:\sigma_1, \dots, x_n:\sigma_n \vdash \varphi \text{ prop}}{\Gamma \vdash \lambda(\vec{x}:\vec{\sigma}). \varphi : (\vec{\sigma}) \rightarrow \text{Prop}}
\end{array}$$

Figure 1: Well-formedness of set types, propositions, kinds and predicate expressions. In the rule for $J(\vec{a})$ (and analogously for $q(\vec{a})$ and index telescopes), $\Gamma \vdash \vec{a} : \vec{\sigma}$ abbreviates $\Gamma \vdash a_i : \sigma_i$ for all i ; since index types are closed, no substitution occurs in the telescope.

datatype and predicate declarations must be well formed in the respective telescopes, and a definition $c := t : \sigma$ (resp. $c := \pi : \varphi$) requires $\vdash \sigma \text{ type}$ and $\vdash t : \sigma$ (resp. $\vdash \varphi \text{ prop}$ and $\vdash \pi : \varphi$) over the prefix.

2.4 Reduction and conversion

Definition 2.4 (Reduction). The relation $\rightarrow$ on terms is the contextual closure — in all term, proof, type, proposition and predicate positions — of the following root rules:

$$\begin{array}{l}
(\lambda x:\sigma. t) a \rightarrow t[a/x] \quad (\lambda p:\varphi. t) \pi \rightarrow t[\pi/p] \\
\text{fix } f:T. t \rightarrow t[\text{fix } f:T. t/f] \quad \text{val}(\text{refine}(t, \pi)) \rightarrow t \\
c \rightarrow t \quad \text{if } (c := t : \sigma) \in \mathbb{E} \quad \text{cast}(\text{refl}_{\sigma}(a); z.\tau; t) \rightarrow t \\
\text{case}(c_i(\vec{a}); z.\tau; \{\dots\}) \rightarrow t_i[\vec{a}/\vec{y}_i] \\
\text{split}(\langle \pi_1, \pi_2 \rangle; p_1 p_2. t) \rightarrow t[\pi_1/p_1][\pi_2/p_2]
\end{array}$$

Source *conversion* $\simeq$ is the least congruence (on each syntactic category, closing under all formers) containing $\rightarrow$.

Reduction is untyped and unrestricted; in particular fixpoint unfolding is not value-restricted. We prove no metatheoretic properties of $\rightarrow$ itself (no confluence or normalisation of the source is needed anywhere in this paper):

$$\begin{array}{c}
\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \quad \frac{(c := t : \sigma) \in \mathbb{E}}{\Gamma \vdash c : \sigma} \quad \frac{\Gamma \vdash t : \Pi x : \sigma. \tau \quad \Gamma \vdash a : \sigma}{\Gamma \vdash ta : \tau[a/x]} \\
\frac{\Gamma \vdash t : \varphi \rightarrow \tau \quad \Gamma \vdash \pi : \varphi}{\Gamma \vdash t\pi : \tau} \quad \frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash \lambda x : \sigma. t : \Pi x : \sigma. \tau} \quad \frac{\Gamma, p : \varphi \vdash t : \tau}{\Gamma \vdash \lambda p : \varphi. t : \varphi \rightarrow \tau} \\
\frac{c : (\sigma_1, \dots, \sigma_r) \in I \quad \Gamma \vdash a_i : \sigma_i \ (1 \leq i \leq r)}{\Gamma \vdash c(\vec{a}) : I} \quad \frac{\Gamma \vdash t : \{x : \sigma \mid \varphi\}}{\Gamma \vdash \text{val}(t) : \sigma} \\
\frac{\Gamma \vdash a : I \quad \Gamma, z : I \vdash \tau \text{ type} \quad \Gamma, \vec{y}_i : \vec{\sigma}_i \vdash t_i : \tau[c_i(\vec{y}_i)/z] \ (all \ i)}{\Gamma \vdash \text{case}(a; z. \tau; \{c_i(\vec{y}_i) \Rightarrow t_i\}_i) : \tau[a/z]} \\
\frac{\Gamma \vdash \{x : \sigma \mid \varphi\} \text{ type} \quad \Gamma \vdash t : \sigma \quad \Gamma \vdash \pi : \varphi[t/x]}{\Gamma \vdash \text{refine}(t, \pi) : \{x : \sigma \mid \varphi\}} \quad \frac{\Gamma \vdash \pi : \perp \quad \Gamma \vdash \sigma \text{ type}}{\Gamma \vdash \text{efq}(\pi; \sigma) : \sigma} \\
\frac{\Gamma \vdash \pi : \varphi_1 \wedge \varphi_2 \quad \Gamma, p_1 : \varphi_1, p_2 : \varphi_2 \vdash t : \tau}{\Gamma \vdash \text{split}(\pi; p_1 p_2. t) : \tau} \quad \frac{\Gamma \vdash t : \sigma \quad \sigma \simeq \sigma' \quad \Gamma \vdash \sigma' \text{ type}}{\Gamma \vdash t : \sigma'} \\
\frac{\Gamma \vdash \pi : a =_\sigma b \quad \Gamma, z : \sigma \vdash \tau \text{ type} \quad \Gamma \vdash t : \tau[a/z]}{\Gamma \vdash \text{cast}(\pi; z. \tau; t) : \tau[b/z]} \\
\frac{\begin{array}{c} T = \Pi x_1 : \sigma_1 \dots \Pi x_n : \sigma_n. \tau \quad \sigma_k = I \quad \Gamma \vdash T \text{ type} \\ t = \lambda \vec{x} : \vec{\sigma}. \text{case}(x_k; z. \tau[z/x_k]; \{c_i(\vec{y}_i) \Rightarrow b_i\}_i) \\ \Gamma, f : T, \vec{x} : \vec{\sigma}, \vec{y}_i : \vec{\sigma}'_i \vdash b_i : \tau[c_i(\vec{y}_i)/x_k] \ (all \ i) \quad \text{grd}_k(f; \vec{y}_i^{\text{rec}}; b_i) \ (all \ i) \end{array}}{\Gamma \vdash \text{fix } f : T. t : T}
\end{array}$$

Figure 2: Typing of terms. In the fix rule, x_k is the designated *guard argument*; $\vec{y}_i^{\text{rec}} \subseteq \vec{y}_i$ denotes the recursive argument positions of c_i (those of type I); and the *guard condition* $\text{grd}_k(f; V; b)$ requires every occurrence of f in b to occur in a subterm of the form $f a_1 \dots a_n$ with $a_k \in V$. We further require $f, \vec{x}, \vec{y}_i$ not to be shadowed in b_i .

$\simeq$ enters only through the conversion typing rules, and the semantics of Section 5 is invariant under it (Lemma 5.6).

Lemma 2.5 (Rigidity of type formers). *If $\sigma \simeq \sigma'$ then σ and σ' have the same outermost former: both are the same datatype name, or both are $\Pi x : p. \tau$ -types whose domains and codomains are respectively convertible, or both are $\varphi \rightarrow \tau$ -types with convertible components, or both are refinement types with convertible components. The analogous statement holds for propositions.*

Proof. No root rule of Definition 2.4 applies at the root of a set type or a proposition — the root rules rewrite terms and proofs only — so every $\rightarrow$ -step inside a type or proposition happens strictly below its outermost former, or below the outermost former of one of its component types and propositions. The claim follows by induction on the length of the conversion path, observing that a single step preserves the outermost former and relates components pointwise. $\square$

Lemma 2.6 (Substitution). *All judgements admit substitution: if $\Gamma, x:\sigma, \Gamma' \vdash \mathcal{J}$ and $\Gamma \vdash a : \sigma$, then $\Gamma, \Gamma'[a/x] \vdash \mathcal{J}[a/x]$, and analogously for proof variables ($\Gamma \vdash \pi : \varphi$) and predicate variables ($\Gamma \vdash P : K$).*

Proof. Routine simultaneous induction on the second derivation, using that $\simeq$ is closed under substitution (each root rule is) and that substitution into predicate atoms $q(\vec{a})[P/q] = P \cdot \vec{a}$ is given by *predicate application*: for $P = \lambda(\vec{x}:\vec{\sigma}).\varphi$ we set $P \cdot \vec{a} = \varphi[\vec{a}/\vec{x}]$. Predicate application terminates: φ cannot contain q (predicate expressions are closed with respect to the variable being substituted after α -renaming), so the substitution $\cdot[P/q]$ is defined by structural recursion on the phrase substituted into, with the atom case delegating to ordinary term substitution inside φ . $\square$

2.5 The shallow fragment

Definition 2.7 (Shallow fragment $\mathfrak{S}$). A proposition is *shallow* if it contains no subformula of the form $\forall q.K.\varphi$ and no atom $q(\vec{a})$ with q a predicate variable; a set type is shallow if all propositions occurring in it (refinement payloads and premises) are shallow. We write $\mathfrak{S}$ for the class of shallow propositions and set types.

Every proposition and type built from datatypes, refinements, inductive predicates, equality and the first-order connectives and quantifiers is shallow; only genuine predicate quantification is excluded. The translation of Section 4 is defined on $\mathfrak{S}$; proofs *inside* the environment may use the full calculus, including predicate quantification — the semantics of Section 5 covers it — but a lemma whose *statement* leaves $\mathfrak{S}$ contributes no premise axiom, a definition whose type leaves $\mathfrak{S}$ contributes no specification axiom, and goals must be shallow and closed. We further require the *declarations* of $\mathbb{E}$ (datatype constructor argument types; inductive-predicate index types, clause telescopes, premises and indices) to lie in $\mathfrak{S}$, since the translation must emit structural axioms for them.

2.6 Relation to CIC

CIC^- is designed as the image of a first-order, monomorphic slice of CIC under analyses that are part of the CoqHammer translation pipeline or of program extraction:

- *Stratification.* CIC’s sort analysis (is this subterm a proof? a proposition? a set-level term?) is decidable and implemented (proposition detection in the translator; [CK18, §4]). CIC^- bakes its outcome into the grammar. Proof irrelevance, which the translation assumes globally, is not needed in the conversion rule of CIC^- (conversion never rewrites inside a proposition’s proof in a way visible to the semantics); it reappears as a property of the *model* (Section 5).

- *Refinement types.* $\text{sig } A \ P$ with its constructor `exist` and projections is an ordinary inductive type in CIC; the classification of inductive instances by propositional content (empty / propositional singleton / refinement / regular) identifies it, per instance, with the primitive $\{x:A \mid P x\}$ of CIC^- — this is Letouzey’s informative-singleton unboxing [Let04, §4.3] applied at the source. Instances of `prod/sigT` with one propositional component classify the same way.
- *Singleton eliminations.* CIC permits matches on `Prop`-sorted scrutinees in informative positions only for empty and singleton propositional inductives; these appear in CIC^- as `efq(;;)`, `split(;;)` and `cast(;;)` ($\perp$, $\wedge$, $=$).
- *Monomorphism.* CIC^- has no type variables and no parameterised inductives: each datatype is declared at a ground instance (`list nat` is a datatype of its own). Heuristic monomorphisation — instantiating polymorphic definitions at the type instances occurring in a problem — is an independent translation stage studied separately; this paper treats the post-monomorphisation fragment.
- *Structural fixpoints.* The `fix` former captures structural recursion with a designated decreasing argument, recursion on immediate constructor subterms, and a case-split body — the common shape of `Fixpoint` definitions. Deep pattern guards, nested and mutual recursion, and well-founded (`Acc`-based) recursion are excluded; Section 10 discusses all four, the last being a genuine semantic boundary rather than a presentational one.
- *Prop-level logic.* The primitive connectives of CIC^- correspond to the inductive definitions of `True`, `False`, `and`, `or`, `ex` and `eq` in CIC; identifying them with primitives is harmless because their CIC induction principles are exactly the elimination rules of Figure 3.

3 The intermediate language and program extraction

The first half of the translation is Letouzey-style program extraction: an erasure of CIC^- terms into an untyped functional calculus $\lambda_\square$. The calculus and the erasure function follow [Let04, Ch. 2], with two simplifications licensed by our setting. First, since the target of the overall translation is an equational theory rather than a programming language with an evaluation order, erased arguments can simply be *dropped* instead of being replaced by a dummy constant with dummy abstractions retained: the evaluation-order problems that force Letouzey to keep dummy λ -abstractions [Let04, §2.1]

concern strict evaluation of extracted code, and no code is run here. (The same simplification would be unsound for well-founded recursion, which is excluded from CIC^- ; see Section 10.3.) Second, the singleton eliminations and refinement unboxing are already primitive in CIC^- , so their erasure clauses are structural.

3.1 The calculus $\lambda_\square$

Definition 3.1 (Syntax of $\lambda_\square$). Fix the datatype constructors $\mathbf{c}$ (with their arities r) and the term-definition constants c of a CIC^- environment $\mathbb{E}$, and a distinguished constant $\square$ (“dead code”). The terms of $\lambda_\square$ are

$$e ::= x \mid c \mid \square \mid \lambda x. e \mid e e' \mid \mathbf{c}(e_1, \dots, e_r) \mid \text{case}(e; \{\mathbf{c}_i(\vec{x}_i) \Rightarrow e_i\}_i) \mid \text{fix } f. e$$

where constructors are fully applied and a case expression carries one branch for each constructor of the datatype of its constructors.

Definition 3.2 (Reduction of $\lambda_\square$). The relation $\rightarrow_\square$ is the contextual closure of:

$$\begin{aligned} (\lambda x. e) e' &\rightarrow_\square e[e'/x] & (\beta) \\ \text{case}(\mathbf{c}_i(\vec{e}); \{\mathbf{c}_j(\vec{x}_j) \Rightarrow e_j\}_j) &\rightarrow_\square e_i[\vec{e}/\vec{x}_i] & (\iota) \\ \text{fix } f. e &\rightarrow_\square e[\text{fix } f. e/f] & (\mu) \\ c &\rightarrow_\square \mathcal{E}(t) \quad \text{if } (c := t : \sigma) \in \mathbb{E} & (\delta) \end{aligned}$$

where $\mathcal{E}$ is the erasure of Definition 3.5 (the δ -rule is well defined because the body of c mentions only constants declared before c). We write $=_\square$ for the least congruence containing $\rightarrow_\square$ ($\lambda_\square$ -conversion).

The four rule families have pairwise distinct, non-overlapping, left-linear redex shapes with rigid heads; $\lambda_\square$ is an orthogonal higher-order rewrite system. We prove confluence directly:

Theorem 3.3 (Confluence of $\lambda_\square$). *The relation $\rightarrow_\square$ is confluent: if $e \rightarrow_\square^* e_1$ and $e \rightarrow_\square^* e_2$ then there is e' with $e_1 \rightarrow_\square^* e'$ and $e_2 \rightarrow_\square^* e'$. Consequently, $e_1 =_\square e_2$ if and only if e_1 and e_2 have a common $\rightarrow_\square^*$ -reduct.*

Proof. By the Tait–Martin–Löf parallel-reduction method in Takahashi’s formulation; the full proof is in Appendix A. Alternatively, $\lambda_\square$ is an orthogonal combinatory reduction system, so confluence follows from Klop’s general theorem [Klo80]. $\square$

Corollary 3.4 (Constructor discrimination and injectivity). *Let $\mathbf{c}, \mathbf{d}$ be constructors.*

1. *If $\mathbf{c}(\vec{e}) =_\square \mathbf{d}(\vec{e}')$ then $\mathbf{c} = \mathbf{d}$ and $e_i =_\square e'_i$ for all i .*

2. An application $e_1 e_2$ in which e_1 is normal and not a λ -abstraction (for instance $e_1 = c(\vec{e})$ with $\vec{e}$ normal) is not convertible to any constructor form $d(\vec{e}')$.

Proof. (1) Constructor-headed terms only reduce internally: every reduct of $c(\vec{e})$ has the form $c(\vec{e}'')$ with $e_i \rightarrow_{\square}^* e_i''$. By Theorem 3.3, $c(\vec{e})$ and $d(\vec{e}')$ have a common reduct, which is then simultaneously c -headed and d -headed, forcing $c = d$; its arguments are common reducts of the respective e_i, e_i' . (2) Such an application is $\rightarrow_{\square}$ -normal apart from reductions inside e_1 and e_2 , which preserve the application shape (the head stays normal and non- λ , so no β , ι , μ or δ redex ever appears at the root); a common reduct with a constructor form is impossible as in (1). $\square$

3.2 Erasure

Definition 3.5 (Erasure). The function $\mathcal{E}$ from CIC^- terms to $\lambda_{\square}$ terms is defined by structural recursion:

$$\begin{aligned}
\mathcal{E}(x) &= x & \mathcal{E}(c) &= c \\
\mathcal{E}(ta) &= \mathcal{E}(t) \mathcal{E}(a) & \mathcal{E}(t\pi) &= \mathcal{E}(t) \\
\mathcal{E}(\lambda x:\sigma.t) &= \lambda x.\mathcal{E}(t) & \mathcal{E}(\lambda p:\varphi.t) &= \mathcal{E}(t) \\
\mathcal{E}(c(\vec{a})) &= c(\mathcal{E}(\vec{a})) & \mathcal{E}(\text{fix } f:T.t) &= \text{fix } f.\mathcal{E}(t) \\
\mathcal{E}(\text{case}(a; z.\tau; \{c_i(\vec{y}_i) \Rightarrow t_i\}_i)) &= \text{case}(\mathcal{E}(a); \{c_i(\vec{y}_i) \Rightarrow \mathcal{E}(t_i)\}_i) \\
\mathcal{E}(\text{refine}(t, \pi)) &= \mathcal{E}(t) & \mathcal{E}(\text{val}(t)) &= \mathcal{E}(t) \\
\mathcal{E}(\text{split}(\pi; p_1 p_2.t)) &= \mathcal{E}(t) & \mathcal{E}(\text{cast}(\pi; z.\tau; t)) &= \mathcal{E}(t) \\
\mathcal{E}(\text{efq}(\pi; \sigma)) &= \square
\end{aligned}$$

Note what the clauses say: propositional arguments and abstractions vanish without residue; refinement values unbox to their informative component; matches on conjunctions and transports along equations collapse to the translated branch; $\perp$ -elimination is dead code. Proofs are never inspected — $\mathcal{E}$ does not recurse into the category of proofs at all — which is the syntactic form of proof irrelevance.

Because $\mathcal{E}$ ignores proofs, types and annotations, the following two lemmas are direct.

Lemma 3.6 (Erasure commutes with substitution). $\mathcal{E}(t[a/x]) = \mathcal{E}(t)[\mathcal{E}(a)/x]$; $\mathcal{E}(t[\pi/p]) = \mathcal{E}(t)$; $\mathcal{E}(t[P/q]) = \mathcal{E}(t)$.

Proof. Induction on t . For the first equation the only interesting cases are the ones in which t mentions x : the variable case is immediate, and all other cases follow from the induction hypothesis because $\mathcal{E}$ is homomorphic in exactly the informative positions and substitution acts pointwise on them. For the second equation, observe that p can occur in t only inside proof

positions or annotations, which $\mathcal{E}$ discards; similarly for the third (q occurs only inside propositions, predicate positions and annotations). $\square$

Lemma 3.7 (Erasure simulates reduction). *If $t \rightarrow u$ then $\mathcal{E}(t) \rightarrow_{\square} \mathcal{E}(u)$ or $\mathcal{E}(t) = \mathcal{E}(u)$. Consequently $t \simeq u$ implies $\mathcal{E}(t) =_{\square} \mathcal{E}(u)$.*

Proof. By induction on the derivation of $t \rightarrow u$. For root steps:

- $(\lambda x:\sigma.t) a \rightarrow t[a/x]$: the erasure of the left side is $(\lambda x.\mathcal{E}(t)) \mathcal{E}(a)$, which β -reduces to $\mathcal{E}(t)[\mathcal{E}(a)/x] = \mathcal{E}(t[a/x])$ by Lemma 3.6.
- $(\lambda p:\varphi.t) \pi \rightarrow t[\pi/p]$: both sides erase to $\mathcal{E}(t)$ by Lemma 3.6; zero steps.
- ι : erases to an ι -step, using Lemma 3.6 for the branch instantiation.
- μ (fixpoint unfolding) and δ : erase to the corresponding $\lambda_{\square}$ steps, again via Lemma 3.6 (for δ , by the definition of the δ -rule of $\lambda_{\square}$).
- the singleton rules $\text{val}(\text{refine}(t, \pi)) \rightarrow t$ and $\text{cast}(\text{refl}; z.\tau; t) \rightarrow t$ and $\text{split}(\langle \pi_1, \pi_2 \rangle; p_1 p_2.t) \rightarrow t[\pi_1, \pi_2/p_1, p_2]$: both sides erase to $\mathcal{E}(t)$ (using Lemma 3.6 for the proof substitutions); zero steps.

For congruence steps: a step inside an informative position maps to a step (or equality) in the corresponding position of the erasure by the induction hypothesis; a step inside a proof, type, proposition or annotation position leaves the erasure unchanged. The consequence for $\simeq$ follows because $=_{\square}$ is reflexive, symmetric, transitive and contains $\rightarrow_{\square}$. $\square$

Remark 3.8. Lemma 3.7 is the equational shadow of extraction correctness [Let04, §2.3]: source computation is simulated by target computation. The semantic half of extraction correctness — the extracted term *realizes* its type — becomes, in our setting, the fundamental lemma of Section 6 (Lemma 6.6), with Letouzey’s simulation predicates $\llbracket \tau \rrbracket$ replaced by the interpretations $\llbracket \tau \rrbracket_{\rho}$ of Section 5.

4 The translation

This section defines the target theory. Throughout, fix a well-formed definitional environment $\mathbb{E}$ whose declarations lie in $\mathfrak{S}$ (Definition 2.7) and a closed shallow goal proposition G .

The target logic is classical first-order logic with equality $\approx$; we write $\vdash_{\text{FOL}}$ for provability in any standard sound and complete proof calculus [Sho67]. We assume the propositional constants $\top$ and $\perp$ are available in the target language (they can be read as abbreviations).

4.1 Signature

Definition 4.1 (Signature $\Sigma_{\mathbb{E}}$). The signature $\Sigma_{\mathbb{E}}$ contains:

- a unary predicate symbol $\widehat{I}$ (the *guard predicate*) for each datatype I of $\mathbb{E}$;
- a k -ary predicate symbol $\widehat{J}$ for each inductive predicate $J : (\sigma_1, \dots, \sigma_k) \rightarrow \text{Prop}$ of $\mathbb{E}$;
- an r -ary function symbol $\widehat{c}$ for each constructor $c : (\sigma_1, \dots, \sigma_r)$ of a datatype of $\mathbb{E}$;
- a function symbol $\widehat{c}$ of arity a_c , and if $a_c > 0$ additionally a constant $\widehat{c}^0$ (the *pointer* of c), for each term definition $c := t : \tau$ of $\mathbb{E}$, where the *erasure arity* a_c is determined from $\mathcal{E}(t)$ in Definition 4.3;
- a binary function symbol $@$ (written infix and associating to the left), and a constant $\odot$ (the image of dead code $\square$);
- countably many *auxiliary* function symbols, produced by the compilation procedure below. As in [Cza18, §4] we assume injective naming functions $\Lambda(\cdot)$, $X(\cdot)$, $\Phi(\cdot)$ from $\lambda_{\square}$ terms (up to renaming of free variables) to auxiliary symbols, so that the symbol lifted for a given term is unique and terms differing only in variable names share their symbol (hash-consing).

Source term variables double as first-order variables.

There is *no* binary inhabitation predicate in $\Sigma_{\mathbb{E}}$: the only typing-flavoured atoms are the per-datatype unary guards $\widehat{I}(\cdot)$, and they occur only at atomic-type leaves of the translation.

4.2 Compilation of erased programs

The compilation turns a closed $\lambda_{\square}$ program into defining equations with constructor patterns. It is presented as a recursive procedure that *emits* first-order axioms and, for every symbol f it introduces, records a *witness*: a $\lambda_{\square}$ term e_f with free variables among the closure parameters of f , used by the model construction of Section 5 and nowhere else.

Compilation carries a finite map θ assigning to certain $\lambda_{\square}$ variables (fix-bound recursion variables) a *template* $(f, \vec{w}, m)$: an $(|\vec{w}| + m)$ -ary symbol f , a list of first-order terms $\vec{w}$, and an argument count m .

Definition 4.2 (Term translation $\mathcal{C}_{\theta}$). For a $\lambda_{\square}$ term e whose free variables are first-order variables or θ -bound, the first-order term $\mathcal{C}_{\theta}(e)$ is defined by recursion on e :

- $\mathcal{C}_\theta(x) = x$ for an ordinary variable; $\mathcal{C}_\theta(\Box) = \odot$; $\mathcal{C}_\theta(\mathbf{c}(\vec{e})) = \widehat{\mathbf{c}}(\mathcal{C}_\theta(\vec{e}))$.
- For an application spine $e = h e_1 \cdots e_j$ ($j \geq 0$, h not an application, e itself not of the previous forms):
 - if $h = g$ with $\theta(g) = (f, \vec{w}, m)$: by the guard condition $j \geq m$, and $\mathcal{C}_\theta(e) = f(\vec{w}, \mathcal{C}_\theta(e_1), \dots, \mathcal{C}_\theta(e_m)) @ \mathcal{C}_\theta(e_{m+1}) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if $h = c$ is a term-definition constant: if $j \geq a_c$ then $\mathcal{C}_\theta(e) = \widehat{c}(\mathcal{C}_\theta(e_1), \dots, \mathcal{C}_\theta(e_{a_c})) @ \cdots @ \mathcal{C}_\theta(e_j)$, and otherwise $\mathcal{C}_\theta(e) = \widehat{c}^0 @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if $h = x$ is a variable: $\mathcal{C}_\theta(e) = x @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$;
 - if h is a λ -abstraction, case or fix (a *complex head*, necessarily with $j \geq 0$): let u be its *value translation* defined below; then $\mathcal{C}_\theta(e) = u @ \mathcal{C}_\theta(e_1) @ \cdots @ \mathcal{C}_\theta(e_j)$.
- Value translations of complex heads h with $\vec{w} = \text{FV}(h)$ (ordered):
 - $h = \lambda x. e_0$: let $f = \Lambda(h)$, an $|\vec{w}|$ -ary symbol with witness $e_f = h$; run $\text{B}_\theta(f(\vec{w}); h)$; the value is $f(\vec{w})$.
 - $h = \text{case}(s; \vec{b}r)$: let $g = \text{X}(\lambda z. \text{case}(z; \vec{b}r))$ for fresh z , of arity $1 + |\vec{w}'|$ with $\vec{w}' = \text{FV}(\vec{b}r)$, with witness $e_g = \lambda z. \text{case}(z; \vec{b}r)$; run $\text{B}_\theta(g(z, \vec{w}'); \text{case}(z; \vec{b}r))$; the value is $g(\mathcal{C}_\theta(s), \vec{w}')$.
 - $h = \text{fix } g_0. \lambda z_1 \dots \lambda z_m. e_0$ (maximal prefix): let $f = \Phi(h)$ of arity $|\vec{w}| + m$ with witness $e_f = h$, and $f^0 = \Phi^0(h)$ of arity $|\vec{w}|$ with witness $e_{f^0} = h$; emit the pointer axiom $\forall \vec{w} \vec{z}. f^0(\vec{w}) @ z_1 @ \cdots @ z_m \approx f(\vec{w}, \vec{z})$; run $\text{B}_{\theta'}(f(\vec{w}, \vec{z}); e_0)$ where $\theta' = \theta[g_0 \mapsto (f, \vec{w}, m)]$; the value is $f^0(\vec{w})$.

Definition 4.3 (Body compilation B). $\text{B}_\theta(L; e)$ takes a first-order term L (the *left-hand side*, linear in its variables, whose variables include $\text{FV}(e)$ minus the θ -bound ones) and a $\lambda_\Box$ term e , and emits axioms:

1. If $e = \lambda x. e'$: run $\text{B}_\theta(L @ x; e')$.
2. If $e = \text{case}(x; \{\mathbf{c}_i(\vec{z}_i) \Rightarrow b_i\}_i)$ with x a variable of L : for each i , with $\vec{z}_i$ fresh, run $\text{B}_\theta(L[\widehat{\mathbf{c}}_i(\vec{z}_i)/x]; b_i[\mathbf{c}_i(\vec{z}_i)/x])$ — the left-hand side is refined by the (hatted) constructor pattern, and the remaining occurrences of the scrutinee variable in the branch are replaced by the (source) constructor pattern.
3. If $e = \text{case}(s; \vec{b}r)$ with s not a variable of L : let $g = \text{X}(\lambda z. \text{case}(z; \vec{b}r))$ with closure $\vec{w}' = \text{FV}(\vec{b}r)$ and witness as in Definition 4.2; emit

$$\forall. L \approx g(\mathcal{C}_\theta(s), \vec{w}')$$

(universal closure over the variables of L), and run $\text{B}_\theta(g(z, \vec{w}'); \text{case}(z; \vec{b}r))$ for fresh z .

4. If $e = \text{fix } g_0.e'$: compute the value translation $u = f^0(\vec{w})$ of e as in Definition 4.2 (which also compiles e); emit $\forall. L \approx u$.
5. Otherwise (e not a λ , case or fix at the root): emit $\forall. L \approx \mathcal{C}_\theta(e)$.

For a term definition $c := t : \tau$ of $\mathbb{E}$, compilation is initialised as follows on $e = \mathcal{E}(t)$ (a closed $\lambda_\square$ term):

- if $e = \text{fix } g_0.\lambda z_1 \dots \lambda z_m.e_0$ (maximal prefix): set $a_c := m$ and run $B_{[g_0 \mapsto (\hat{c}, (), m)]}(\hat{c}(\vec{z}); e_0)$;
- otherwise, with $e = \lambda z_1 \dots \lambda z_m.e_0$ (maximal prefix, possibly $m = 0$): set $a_c := m$ and run $B_\emptyset(\hat{c}(\vec{z}); e_0)$.

The witness of $\hat{c}$ and of $\hat{c}^0$ is the $\lambda_\square$ constant c ; if $a_c > 0$, the pointer axiom $\forall \vec{z}. \hat{c}^0 @ z_1 @ \dots @ z_{a_c} \approx \hat{c}(\vec{z})$ is emitted.

Lemma 4.4 (Compilation terminates). *The procedure of Definitions 4.2 and 4.3 terminates on every input, and (with the naming functions of Definition 4.1) emits a finite set of axioms for a given environment and goal.*

Proof. Define the size $|e|$ of a $\lambda_\square$ term as its number of syntax nodes. Every recursive call of B or $\mathcal{C}$ is on a term of strictly smaller size, with one exception: rule (3) of B passes from $\text{case}(s; \vec{b}r)$ to $\text{case}(z; \vec{b}r)$. Since s is not a variable in rule (3), $|s| > 1 = |z|$, so the size still strictly decreases. In rule (2) the branch b_i has its scrutinee variable replaced by a constructor pattern of fresh *variables*; the substituted occurrences of x in b_i are variable occurrences replaced by terms of size $1 + r$ — to make the measure genuinely decrease we count the size of the *pair* (number of case/fix nodes of e , node count), ordered lexicographically: rule (2) consumes the outer case node and passes to a proper subterm b_i of e up to a variable-for-pattern replacement that does not create case, λ or fix nodes; rules (1), (3), (4), (5) and all $\mathcal{C}$ recursions also strictly decrease the pair. Finiteness follows since each emitted axiom is attached to a symbol, each symbol to a $\lambda_\square$ term occurring in the (finite) compilation tree, and hash-consing merges repeated liftings. $\square$

Remark 4.5 (Shape of the equations). All axioms emitted by compilation are universally quantified equations $L \approx R$ whose left-hand sides are linear terms of the form $f(\vec{\pi}) @ x_1 @ \dots @ x_k$ with $\vec{\pi}$ constructor patterns and variables. Grouped by their head symbol, the left-hand sides arising from one case tree are pairwise non-overlapping (they refine disjoint constructor patterns), so the emitted equations form an *orthogonal* first-order rewrite system when oriented left to right [BN98]. This property is not used in the soundness proof, but it is the operational point of the exercise: the equations are orientable rewrite rules, usable by superposition provers for demodulation and by SMT solvers as triggers (Section 7.3).

Example 4.6 (Basic definitions). Let nat have constructors $\text{O} : ()$ and $\text{S} : (\text{nat})$, and bool have $\text{true}, \text{false} : ()$. Writing the hatted symbols without hats for readability:

1. $\text{mypred} := \lambda x:\text{nat}. \text{case}(x; _.\text{nat}; \text{O} \Rightarrow \text{O} \mid \text{S}(y) \Rightarrow y)$ compiles to

$$\text{mypred}(\text{O}) \approx \text{O}, \quad \forall y. \text{mypred}(\text{S}(y)) \approx y.$$

2. $\text{add} := \text{fix } f:\Pi n m:\text{nat}. \text{nat}. \lambda n m. \text{case}(n; _.\text{nat}; \text{O} \Rightarrow m \mid \text{S}(y) \Rightarrow \text{S}(f y m))$ compiles to

$$\forall m. \text{add}(\text{O}, m) \approx m, \quad \forall y m. \text{add}(\text{S}(y), m) \approx \text{S}(\text{add}(y, m)).$$

3. Nested matches flatten into deeper patterns: $g x := \text{case}(x; ; \text{O} \Rightarrow \text{O} \mid \text{S}(y) \Rightarrow \text{case}(y; ; \text{O} \Rightarrow \text{S}(\text{O}) \mid \text{S}(z) \Rightarrow z))$ compiles to $g(\text{O}) \approx \text{O}$, $g(\text{S}(\text{O})) \approx \text{S}(\text{O})$ and $\forall z. g(\text{S}(\text{S}(z))) \approx z$.
4. A compound scrutinee is flattened through an auxiliary symbol: $k b := \text{case}(\text{negb } b; ; \text{true} \Rightarrow \text{O} \mid \text{false} \Rightarrow \text{S}(\text{O}))$ compiles to

$$\forall b. k(b) \approx g'(\text{negb}(b)), \quad g'(\text{true}) \approx \text{O}, \quad g'(\text{false}) \approx \text{S}(\text{O}),$$

where g' is the auxiliary symbol lifted for the case expression.

No axiom in this list carries a guard: compare Section 7.

4.3 Specification extraction

Definition 4.7 (Guard formulas and proposition translation). For every shallow set type σ and first-order term u , the *guard formula* $\mathcal{G}_\sigma(u)$, and for every shallow proposition φ , the first-order formula $\mathcal{F}(\varphi)$, are defined by mutual structural recursion:

$$\begin{aligned} \mathcal{G}_I(u) &= \hat{I}(u) \\ \mathcal{G}_{\Pi x:\sigma.\tau}(u) &= \forall x. \mathcal{G}_\sigma(x) \rightarrow \mathcal{G}_\tau(u @ x) \\ \mathcal{G}_{\varphi \rightarrow \tau}(u) &= \mathcal{F}(\varphi) \rightarrow \mathcal{G}_\tau(u) \\ \mathcal{G}_{\{x:\sigma \mid \varphi\}}(u) &= \mathcal{G}_\sigma(u) \wedge \mathcal{F}(\varphi)[u/x] \\ \mathcal{F}(\top) &= \top, \quad \mathcal{F}(\perp) = \perp, \quad \mathcal{F}(\varphi \circledast \psi) = \mathcal{F}(\varphi) \circledast \mathcal{F}(\psi) \quad (\circledast \in \{\wedge, \vee, \rightarrow\}) \\ \mathcal{F}(\forall x:\sigma.\varphi) &= \forall x. \mathcal{G}_\sigma(x) \rightarrow \mathcal{F}(\varphi) \\ \mathcal{F}(\exists x:\sigma.\varphi) &= \exists x. \mathcal{G}_\sigma(x) \wedge \mathcal{F}(\varphi) \\ \mathcal{F}(a =_\sigma b) &= \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b)) \\ \mathcal{F}(J(\vec{a})) &= \hat{J}(\mathcal{C}(\mathcal{E}(a_1)), \dots, \mathcal{C}(\mathcal{E}(a_k))) \end{aligned}$$

where in the clauses for Π , $\forall$, $\exists$ and refinement the bound source variable is reused as the bound first-order variable, and $\mathcal{C} = \mathcal{C}_\emptyset$ is the term translation of

Definition 4.2 (whose complex-head clauses may emit further axioms). Note that $\mathcal{G}$ and $\mathcal{F}$ are total on $\mathfrak{S}$ and structurally recursive; no normalisation of the source is involved.

Two clauses implement the shallowness requirements discussed in Sections 1 and 8. The refinement clause expands the propositional payload *inline at each occurrence* — positively under $\exists$ and in conclusions, negatively under $\forall$ and in premises — rather than emitting an opaque atom plus an unfolding axiom. The premise clause $\mathcal{G}_{\varphi \rightarrow \tau}(u) = \mathcal{F}(\varphi) \rightarrow \mathcal{G}_{\tau}(u)$ does *not* apply u to anything: erasure has dropped the proof argument, so the specification tracks the pruned arity of the extracted program.

Definition 4.8 (Specification of a constant). For a term definition $c := t : \tau$ with $\tau \in \mathfrak{S}$, the *specification axiom* $\mathcal{S}(c)$ is $S(u_0; \tau)$ where $u_0 = \widehat{c}$ if $a_c = 0$ and otherwise $u_0 = \widehat{c}()$ is the empty application accumulator, and

$$\begin{aligned} S(u; \Pi x : \sigma. \tau') &= \forall x. \mathcal{G}_{\sigma}(x) \rightarrow S(u \cdot x; \tau') \\ S(u; \varphi \rightarrow \tau') &= \mathcal{F}(\varphi) \rightarrow S(u; \tau') \\ S(u; \tau') &= \mathcal{G}_{\tau'}(\bar{u}) \quad \text{if } \tau' \text{ is a datatype or refinement type,} \end{aligned}$$

where $u \cdot x$ appends x to the accumulator if fewer than a_c arguments have been accumulated and otherwise applies with $@$, and $\bar{u}$ closes the accumulator: an under-applied accumulator $\widehat{c}(x_1, \dots, x_j)$ with $j < a_c$ is rendered as the pointer chain $\widehat{c}^0 @ x_1 @ \dots @ x_j$.

Example 4.9 (The running example h). Consider

$$\begin{aligned} h &:= \lambda x y z : \text{nat}. \lambda p : (x =_{\text{nat}} y \wedge y =_{\text{nat}} z). \text{split}(p; p_1 p_2. \text{refine}(z, \text{trans}(p_1, p_2))) \\ h &: \Pi x y z : \text{nat}. (x = y \wedge y = z) \rightarrow \{u : \text{nat} \mid x =_{\text{nat}} u\} \end{aligned}$$

where $\text{trans}(p_1, p_2)$ abbreviates the evident proof of $x =_{\text{nat}} z$ by rew . Erasure gives

$$\mathcal{E}(h) = \lambda x y z. z,$$

(the proof abstraction disappears, $\text{split}(\cdot)$ collapses to its branch, $\text{refine}(\cdot)$ unboxes), so $a_h = 3$ and compilation emits the single equation

$$\forall x y z. \widehat{h}(x, y, z) \approx z.$$

Specification extraction on the type gives

$$\begin{aligned} \mathcal{S}(h) &= \forall x. \text{nat}(x) \rightarrow \forall y. \text{nat}(y) \rightarrow \forall z. \text{nat}(z) \rightarrow \\ &\quad \left((x \approx y \wedge y \approx z) \rightarrow (\text{nat}(\widehat{h}(x, y, z)) \wedge x \approx \widehat{h}(x, y, z)) \right). \end{aligned}$$

Both axioms requested by the design are produced, and the symbols sig , exist , proj1_sig and any inhabitation atom are absent from the output. Note the interplay of the components: the equation is blind to the propositional content (h is extractionally a projection), while the specification carries it; for an underspecified function the situation is reversed.

4.4 The theory $\Delta(\mathbb{E}, G)$

Definition 4.10 (Emitted theory). The first-order theory $\Delta(\mathbb{E}, G)$ consists of the following axioms.

1. For each datatype I with constructors $c_i : (\sigma_{i1}, \dots, \sigma_{ir_i})$:
 - (*constructor typing*) $\forall \vec{y}. \bigwedge_j \mathcal{G}_{\sigma_{ij}}(y_j) \rightarrow \widehat{I}(\widehat{c}_i(\vec{y}))$ for each i ;
 - (*inversion*) $\forall w. \widehat{I}(w) \rightarrow \bigvee_i \exists \vec{y}. \bigwedge_j \mathcal{G}_{\sigma_{ij}}(y_j) \wedge w \approx \widehat{c}_i(\vec{y})$;
 - (*injectivity*) $\forall \vec{y} \vec{y}'. \widehat{c}_i(\vec{y}) \approx \widehat{c}_i(\vec{y}') \rightarrow \bigwedge_j y_j \approx y'_j$ for each i with $r_i > 0$;
 - (*discrimination*) $\forall \vec{y} \vec{y}'. \neg(\widehat{c}_i(\vec{y}) \approx \widehat{c}_j(\vec{y}'))$ for all $i \neq j$.
2. For each inductive predicate J with clauses $k_j : \forall \vec{x} : \vec{\tau}_j. \varphi_{j1} \rightarrow \dots \rightarrow \varphi_{j\ell_j} \rightarrow J(\vec{u}_j)$:
 - (*clause*) $\forall \vec{x}. \bigwedge_i \mathcal{G}_{\tau_{ji}}(x_i) \rightarrow \mathcal{F}(\varphi_{j1}) \rightarrow \dots \rightarrow \mathcal{F}(\varphi_{j\ell_j}) \rightarrow \widehat{J}(\mathcal{C}(\mathcal{E}(\vec{u}_j)))$;
 - (*case analysis*) $\forall \vec{z}. \widehat{J}(\vec{z}) \rightarrow \bigvee_j \exists \vec{x}. \bigwedge_i \mathcal{G}_{\tau_{ji}}(x_i) \wedge \bigwedge_i \mathcal{F}(\varphi_{ji}) \wedge \bigwedge_l z_l \approx \mathcal{C}(\mathcal{E}(u_{jl}))$.
3. For each term definition $c := t : \tau$: the equations, pointer axioms and auxiliary axioms emitted by compiling $\mathcal{E}(t)$ (Definition 4.3), and the specification axiom $\mathcal{S}(c)$ if $\tau \in \mathfrak{S}$.
4. For each lemma $c := \pi : \varphi$ with $\varphi \in \mathfrak{S}$: the *premise axiom* $\mathcal{F}(\varphi)$.
5. The auxiliary axioms emitted by the computation of $\mathcal{F}(G)$ (via the complex-head clauses of $\mathcal{C}$ on terms embedded in G).

The *translation of the problem* $(\mathbb{E}, G)$ is the pair $(\Delta(\mathbb{E}, G), \mathcal{F}(G))$.

Remark 4.11. (i) The case-analysis axiom for an inductive predicate is the formula-level image of case analysis on a derivation — inversion in the proof-assistant sense; full induction is a schema over motives and is not emitted (an ATP would rarely be able to instantiate it; when induction is needed it is the reconstruction backend’s job). (ii) Premise selection operates before translation, so in practice $\Delta(\mathbb{E}, G)$ is generated from the selected slice of the environment; soundness is monotone in the axiom set, so Theorem 6.9 covers every subset of $\Delta(\mathbb{E}, G)$. (iii) The per-datatype inversion axiom is where exhaustiveness of constructors lives; the defining equations of Definition 4.3 make no such claim. This division of labour is the subject of Section 7.

5 Semantics

We now build the structures against which soundness is proved: a *semantics of the source* — proof-irrelevant, classical, and syntactic in the sense that individuals are $\lambda_{\square}$ terms — and a first-order structure $\mathfrak{M}$ over the same individuals interpreting the signature $\Sigma_{\mathbb{E}}$. Throughout, $\mathbb{E}$ is a fixed well-formed definitional environment with $\mathfrak{S}$ declarations.

5.1 The domain

Definition 5.1 (Domain). Let $\mathcal{D}$ be the set of closed $\lambda_{\square}$ terms modulo $=_{\square}$. We write $[e]$ for the class of e . $\mathcal{D}$ is non-empty ($[\square] \in \mathcal{D}$) and carries an application operation $d \cdot d' := [e e']$ for representatives $e \in d$, $e' \in d'$, well defined since $=_{\square}$ is a congruence.

$\mathcal{D}$ deliberately contains *junk*: classes such as $[\mathbf{O} \mathbf{O}]$ (a constructor applied to a constructor) or $[\text{fix } f.f]$ that are the value of no source term of any type. By Corollary 3.4, distinct constructor forms lie in distinct classes, and junk classes like $[\mathbf{O} \mathbf{O}]$ lie outside every constructor form. Junk is what makes the guard analysis of Section 7 non-trivial, and keeping it in the model — rather than quotienting it away — is what lets the model arbitrate which axioms need guards.

Definition 5.2 (Valuations and term denotation). A *valuation* ρ assigns to finitely many term variables elements of $\mathcal{D}$ and to finitely many predicate variables q (of kind $(\sigma_1, \dots, \sigma_n) \rightarrow \text{Prop}$) functions $\mathcal{D}^n \rightarrow \{0, 1\}$. Proof variables are not assigned anything. For a CIC^- term t whose free term variables are assigned by ρ , the *denotation* is

$$\llbracket t \rrbracket_{\rho} := [\mathcal{E}(t)^{\rho}] \in \mathcal{D},$$

where $\mathcal{E}(t)^{\rho}$ substitutes a representative of $\rho(x)$ for each free variable x of $\mathcal{E}(t)$. This is well defined: free variables of $\mathcal{E}(t)$ are free term variables of t (proof and predicate variables do not survive erasure by Definition 3.5), and the class is independent of the choice of representatives because $=_{\square}$ is a congruence. For a $\lambda_{\square}$ term e we likewise write $[e^{\rho}]$.

5.2 Interpretation of types, propositions and predicates

Definition 5.3 (Interpretations). By recursion on the position of declarations in $\mathbb{E}$ and, within each declaration, on the structure of phrases, we define for every set type σ , proposition φ and predicate expression P over a prefix of $\mathbb{E}$, and every valuation ρ covering their free variables:

- a set $\llbracket \sigma \rrbracket_{\rho} \subseteq \mathcal{D}$,
- a truth value $\langle \varphi \rangle_{\rho} \in \{0, 1\}$,

- a function $\llbracket P \rrbracket_\rho : \mathcal{D}^n \rightarrow \{0, 1\}$ for P of kind $(\sigma_1, \dots, \sigma_n) \rightarrow \mathbf{Prop}$,

as follows.

Datatypes. For a datatype I with constructors $c_i : (\sigma_{i1}, \dots, \sigma_{ir_i})$, define stages $\llbracket I \rrbracket^m \subseteq \mathcal{D}$ by $\llbracket I \rrbracket^0 = \emptyset$ and

$$\begin{aligned} \llbracket I \rrbracket^{m+1} = \{ & \llbracket c_i(e_1, \dots, e_{r_i}) \rrbracket \mid i \leq \text{number of constructors,} \\ & [e_j] \in \llbracket \sigma_{ij} \rrbracket \text{ for non-recursive } j, \quad [e_j] \in \llbracket I \rrbracket^m \text{ for recursive } j \}, \end{aligned}$$

which is well defined because the non-recursive σ_{ij} are closed types over the environment prefix preceding I . The stages are increasing in m (induction on m), and we set $\llbracket I \rrbracket = \bigcup_m \llbracket I \rrbracket^m$ and define the *rank* $\text{rk}_I(d)$ of $d \in \llbracket I \rrbracket$ as the least m with $d \in \llbracket I \rrbracket^m$.

Set types.

$$\begin{aligned} \llbracket \Pi x:\sigma. \tau \rrbracket_\rho &= \{ d \in \mathcal{D} \mid \forall e \in \llbracket \sigma \rrbracket_\rho. d \cdot e \in \llbracket \tau \rrbracket_{\rho[x \mapsto e]} \} \\ \llbracket \varphi \rightarrow \tau \rrbracket_\rho &= \{ d \in \mathcal{D} \mid \langle \varphi \rangle_\rho = 1 \implies d \in \llbracket \tau \rrbracket_\rho \} \\ \llbracket \{x:\sigma \mid \varphi\} \rrbracket_\rho &= \{ d \in \llbracket \sigma \rrbracket_\rho \mid \langle \varphi \rangle_{\rho[x \mapsto d]} = 1 \} \end{aligned}$$

The clause for $\varphi \rightarrow \tau$ has no application: it mirrors the erasure of the proof argument (Definition 3.5).

Inductive predicates. For $J : (\sigma_1, \dots, \sigma_k) \rightarrow \mathbf{Prop}$ with clauses $k_j : \forall \vec{x}:\vec{\tau}_j. \varphi_{j1} \rightarrow \dots \rightarrow \varphi_{j\ell_j} \rightarrow J(\vec{u}_j)$, define stages $\langle J \rangle^0 = \emptyset \subseteq \mathcal{D}^k$ and

$$\begin{aligned} \langle J \rangle^{m+1} = \{ & (\llbracket u_{j1} \rrbracket_{\rho_0}, \dots, \llbracket u_{jk} \rrbracket_{\rho_0}) \mid j, \rho_0 = [\vec{x} \mapsto \vec{d}], d_i \in \llbracket \tau_{ji} \rrbracket_{\rho_0}, \\ & \langle \varphi \rangle_{\rho_0} = 1 \text{ for non-rec. premises, } \vec{v}\text{-tuples} \in \langle J \rangle^m \text{ for rec. premises} \}, \end{aligned}$$

where a recursive premise $J(\vec{v})$ contributes the condition $(\llbracket v_1 \rrbracket_{\rho_0}, \dots, \llbracket v_k \rrbracket_{\rho_0}) \in \langle J \rangle^m$. (Premises and index types mention only J -free material and earlier declarations, plus the telescope variables $\vec{x}$, so the right-hand side is well defined.) Set $\langle J \rangle = \bigcup_m \langle J \rangle^m$, with ranks defined as for datatypes.

Propositions.

$$\begin{aligned} \langle \top \rangle_\rho &= 1, \quad \langle \perp \rangle_\rho = 0 \\ \langle \varphi \wedge \psi \rangle_\rho &= \min(\langle \varphi \rangle_\rho, \langle \psi \rangle_\rho), \quad \langle \varphi \vee \psi \rangle_\rho = \max(\langle \varphi \rangle_\rho, \langle \psi \rangle_\rho) \\ \langle \varphi \rightarrow \psi \rangle_\rho &= \max(1 - \langle \varphi \rangle_\rho, \langle \psi \rangle_\rho) \\ \langle \forall x:\sigma. \varphi \rangle_\rho &= \min_{d \in \llbracket \sigma \rrbracket_\rho} \langle \varphi \rangle_{\rho[x \mapsto d]}, \quad \langle \exists x:\sigma. \varphi \rangle_\rho = \max_{d \in \llbracket \sigma \rrbracket_\rho} \langle \varphi \rangle_{\rho[x \mapsto d]} \\ \langle \forall q:K. \varphi \rangle_\rho &= \min_{Q: \mathcal{D}^n \rightarrow \{0,1\}} \langle \varphi \rangle_{\rho[q \mapsto Q]} \\ \langle a =_\sigma b \rangle_\rho &= 1 \text{ iff } \llbracket a \rrbracket_\rho = \llbracket b \rrbracket_\rho \\ \langle J(\vec{a}) \rangle_\rho &= 1 \text{ iff } (\llbracket a_1 \rrbracket_\rho, \dots, \llbracket a_k \rrbracket_\rho) \in \langle J \rangle \\ \langle q(\vec{a}) \rangle_\rho &= \rho(q)(\llbracket a_1 \rrbracket_\rho, \dots, \llbracket a_n \rrbracket_\rho) \end{aligned}$$

(with $\min \emptyset = 1$, $\max \emptyset = 0$).

Predicate expressions. $\llbracket q \rrbracket_\rho = \rho(q)$ and $\llbracket \lambda(\vec{x}:\vec{\sigma}).\varphi \rrbracket_\rho(\vec{d}) = \langle \varphi \rangle_{\rho[\vec{x} \mapsto \vec{d}]}$.

The recursion is well founded: the datatype and predicate stages refer only to earlier declarations and to the (structurally smaller) phrases of their own declaration; all other clauses recurse on proper sub-phrases. The semantics is classical and *proof-irrelevant* by construction: truth values do not depend on proofs, and $\llbracket \cdot \rrbracket$ ignores proof subterms. Note also that the interpretation of equality is untyped — $\langle a =_\sigma b \rangle$ compares classes in $\mathcal{D}$ and does not mention σ — which matches the translation clause $\mathcal{F}(a =_\sigma b) = \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b))$.

Lemma 5.4 (Elementary properties). 1. $\llbracket \sigma \rrbracket_\rho$, $\langle \varphi \rangle_\rho$, $\llbracket P \rrbracket_\rho$ depend only on the restriction of ρ to the free variables of the phrase.

2. (Weakening) They are unchanged by extending ρ on fresh variables.

3. Every $d \in \llbracket I \rrbracket$ has a finite rank, and $d \in \llbracket I \rrbracket^m$ for all $m > \text{rk}_I(d)$; similarly for $\langle J \rangle$.

Proof. (1), (2): routine induction on the phrase. (3): by definition of the union and monotonicity of stages. $\square$

Lemma 5.5 (Semantic substitution). Let a be a term, π a proof and P a predicate expression, with ρ covering the relevant free variables. Then:

1. $\llbracket t[a/x] \rrbracket_\rho = \llbracket t \rrbracket_{\rho[x \mapsto \llbracket a \rrbracket_\rho]}$, and the analogous equations for the substituted type $\sigma[a/x]$, proposition $\varphi[a/x]$ and predicate expression $P[a/x]$.
2. Substitution of proofs is invisible: $\llbracket t[\pi/p] \rrbracket_\rho = \llbracket t \rrbracket_\rho$, $\llbracket \sigma[\pi/p] \rrbracket_\rho = \llbracket \sigma \rrbracket_\rho$, $\langle \varphi[\pi/p] \rangle_\rho = \langle \varphi \rangle_\rho$.
3. $\langle \varphi[P/q] \rangle_\rho = \langle \varphi \rangle_{\rho[q \mapsto \llbracket P \rrbracket_\rho]}$, and analogously for types and terms (on which the substitution acts only through annotations and is invisible).

Proof. (1) For terms: $\mathcal{E}(t[a/x]) = \mathcal{E}(t)[\mathcal{E}(a)/x]$ by Lemma 3.6, and substituting a representative of $\llbracket a \rrbracket_\rho$ for x yields a representative of the same class. For types, propositions and predicates: induction on the phrase, with the term case as base; the binder cases use (2) of Lemma 5.4; the predicate-application case $(q(\vec{a}))[P/q] = P \cdot \vec{a}$ in (3) unfolds $\llbracket P \rrbracket_\rho$ by definition. (2) By Lemma 3.6, erasure discards the substituted proof, and proofs do not occur in the semantic clauses. (3) As in (1). $\square$

Lemma 5.6 (Conversion invariance). If $t \simeq t'$ then $\llbracket t \rrbracket_\rho = \llbracket t' \rrbracket_\rho$. If $\sigma \simeq \sigma'$ (both well formed) then $\llbracket \sigma \rrbracket_\rho = \llbracket \sigma' \rrbracket_\rho$; if $\varphi \simeq \varphi'$ then $\langle \varphi \rangle_\rho = \langle \varphi' \rangle_\rho$.

Proof. For terms: by Lemma 3.7, $t \simeq t'$ implies $\mathcal{E}(t) =_{\square} \mathcal{E}(t')$, hence $\mathcal{E}(t)^\rho =_{\square} \mathcal{E}(t')^\rho$ ($=_{\square}$ is closed under substitution of representatives), i.e. $\llbracket t \rrbracket_\rho = \llbracket t' \rrbracket_\rho$.

For types and propositions, first consider a single step $\sigma \rightarrow \sigma'$. By Lemma 2.5 (and its proof), the step occurs inside a term position (an equation side, a predicate-atom argument, an index) or inside a component type or proposition. Proceed by induction on the structure of σ : in the first case the claim follows from the term statement, since the semantic clauses depend on embedded terms only through $\llbracket \cdot \rrbracket_\rho$; in the second, from the induction hypothesis applied to the component (for binders, at every extension $\rho[x \mapsto d]$, using that the same step relates the instantiated components). The general claim follows by induction on the length of the conversion path. $\square$

5.3 The first-order structure $\mathfrak{M}$

Definition 5.7 (The structure $\mathfrak{M}$). $\mathfrak{M}$ is the first-order $\Sigma_{\mathbb{E}}$ -structure with universe $\mathcal{D}$ and:

- $\approx$ interpreted as identity of classes; $@$ as $\cdot$; $\odot$ as $[\square]$;
- $\hat{I}^{\mathfrak{M}} = \llbracket I \rrbracket$ for each datatype I , and $\hat{J}^{\mathfrak{M}} = \langle J \rangle$ for each inductive predicate J ;
- $\hat{c}^{\mathfrak{M}}(d_1, \dots, d_r) = [c(e_1, \dots, e_r)]$ for representatives $e_i \in d_i$;
- for every symbol f introduced by compilation with closure arity s , extension arity m and witness e_f (a $\lambda_{\square}$ term with free variables $\vec{w} = w_1, \dots, w_s$; see Definitions 4.2 and 4.3):

$$f^{\mathfrak{M}}(d_1, \dots, d_s, d'_1, \dots, d'_m) = [e_f[\vec{e}/\vec{w}] e'_1 \cdots e'_m]$$

for representatives $e_i \in d_i$, $e'_j \in d'_j$. In particular $\hat{c}^{\mathfrak{M}}(\vec{d}) = [c \vec{e}]$ (the $\lambda_{\square}$ constant c applied) and $(\hat{c}^0)^{\mathfrak{M}} = [c]$.

All interpretations are well defined on classes by congruence of $=_{\square}$. We write $u^{\mathfrak{M}, \rho}$ for the denotation in $\mathfrak{M}$ of a first-order term u under an assignment ρ of its variables, and $\mathfrak{M}, \rho \models \phi$ for satisfaction of a first-order formula, in the standard (classical, Tarskian) sense.

Since source term variables double as first-order variables, a valuation ρ (Definition 5.2) restricted to term variables serves as a first-order assignment, and we use the same letter for both.

5.4 Adequacy of the term translation and of compilation

Definition 5.8 (Coherent template maps). Let θ be a template map as in Section 4.2, and σ_{θ} a substitution assigning to each $g \in \text{dom}(\theta)$ a closed $\lambda_{\square}$

term $\sigma_\theta(g)$. Say (θ, σ_θ) is *coherent* for an assignment ρ if for every g with $\theta(g) = (f, \vec{w}, m)$ and all $d_1, \dots, d_m \in \mathcal{D}$ with representatives e_i ,

$$f^{\mathfrak{M}}(\vec{w}^{\mathfrak{M}, \rho}, \vec{d}) = [\sigma_\theta(g) e_1 \cdots e_m].$$

Lemma 5.9 (Adequacy of $\mathcal{C}$). *Let e be a $\lambda_\square$ term whose free variables are first-order variables assigned by ρ or bound by θ , and let (θ, σ_θ) be coherent for ρ . Then*

$$\mathcal{C}_\theta(e)^{\mathfrak{M}, \rho} = [e^{\rho, \sigma_\theta}],$$

where e^{ρ, σ_θ} substitutes representatives of $\rho(x)$ for the first-order variables and $\sigma_\theta(g)$ for the θ -bound variables.

Proof. By induction on e , following the clauses of Definition 4.2. Variables, $\square$ and constructors are immediate. For a spine $h e_1 \cdots e_j$:

- $h = g$ θ -bound: by coherence, $f^{\mathfrak{M}}(\vec{w}^{\mathfrak{M}, \rho}, \mathcal{C}_\theta(e_1)^{\mathfrak{M}, \rho}, \dots) = [\sigma_\theta(g) e_1^{\rho, \sigma_\theta} \cdots]$ using the induction hypothesis on the e_i ; the remaining arguments are appended by $@$, interpreted as $\cdot$.
- $h = c$: $\widehat{c}^{\mathfrak{M}}$ applies the $\lambda_\square$ constant c to the argument classes (Definition 5.7), which is exactly $[(c e_1 \cdots e_j)^{\rho, \sigma_\theta}]$, with the pointer case identical via $(\widehat{c}^0)^{\mathfrak{M}} = [c]$ and $@$.
- h a variable: immediate.
- h complex: by Definition 5.7 the head's value translation denotes $[h^{\rho, \sigma_\theta}]$ — for a λ -lift, $f^{\mathfrak{M}}(\vec{w}^{\mathfrak{M}, \rho}) = [e_f^{\rho, \sigma_\theta}]$ with $e_f = h$, and similarly for case lifts $(g^{\mathfrak{M}}(\mathcal{C}(s)^{\mathfrak{M}}, \dots) = [(\lambda z. \text{case}(z; \vec{br}))^\cdots s^\cdots]$, which β -reduces to $[\text{case}(s; \vec{br})^\cdots]$, using the induction hypothesis for s) and fix lifts (via the pointer's witness); the spine arguments are then appended by $\cdot$.

Note the case for complex heads uses only the *interpretation* of the lifted symbols, not the axioms emitted for them. $\square$

Lemma 5.10 (Soundness of compilation). *Consider any invocation of the compilation procedure on the environment (or on a term embedded in the goal), and let $B_\theta(L; e)$ range over the body-compilation calls it performs. Then:*

1. (Invariant) *For every such call there is a substitution σ_θ such that for every assignment ρ of the variables of L , the pair (θ, σ_θ) is coherent for ρ and $L^{\mathfrak{M}, \rho} = [e^{\rho, \sigma_\theta}]$.*
2. *Every axiom emitted by the procedure holds in $\mathfrak{M}$.*

Proof. By induction on the tree of procedure calls. We first check the invariant at the roots, then that each rule preserves it and that each emission is valid given it.

Roots. For a definition $c := t : \tau$ with $\mathcal{E}(t) = \lambda \vec{z}.e_0$ (no fix): $\widehat{c}(\vec{z})^{\mathfrak{M},\rho} = [c \vec{e}] = [\mathcal{E}(t) \vec{e}]$ (a δ -step) $= [e_0^\rho]$ (by $|\vec{z}|$ β -steps), with $\theta = \emptyset$. For $\mathcal{E}(t) = \text{fix } g_0.\lambda \vec{z}.e_0$: set $\sigma_\theta(g_0) = \mathcal{E}(t)$; coherence holds because $\widehat{c}^{\mathfrak{M}}(\vec{d}) = [c \vec{e}] = [\mathcal{E}(t) \vec{e}]$ by a δ -step; and $\widehat{c}(\vec{z})^{\mathfrak{M},\rho} = [c \vec{e}] = [\mathcal{E}(t) \vec{e}] = [(\lambda \vec{z}.e_0)[\mathcal{E}(t)/g_0] \vec{e}] = [e_0^{\rho, \sigma_\theta}]$ by a μ -step and β -steps. For a λ -lift of $h = \lambda x.e'$ (Definition 4.2), the call is $B_\theta(f(\vec{w}); h)$ and $f(\vec{w})^{\mathfrak{M},\rho} = [h^{\rho, \sigma_\theta}]$ holds by the definition of $f^{\mathfrak{M}}$ (witness h); for a case lift, $g(z, \vec{w}')^{\mathfrak{M},\rho} = [(\lambda z'.\text{case}(z'; \vec{b}r)) \cdots z^\rho] = \square [\text{case}(z; \vec{b}r)^{\rho, \sigma_\theta}]$ by a β -step; for a fix lift $h = \text{fix } g_0.\lambda \vec{z}.e_0$, extend σ_θ with $\sigma_\theta(g_0) = h^{\rho, \sigma_\theta}|_{\vec{w}}$ — the closed instance of h at the representatives of $\vec{w}^{\mathfrak{M},\rho}$; coherence for g_0 is the definition of $f^{\mathfrak{M}}$ via witness h , and the invariant for $B_{\theta'}(f(\vec{w}, \vec{z}); e_0)$ follows by one μ -step and β -steps as in the definition-root case.

Rule (1) ($e = \lambda x.e'$): $L @ x^{\mathfrak{M},\rho} = L^{\mathfrak{M},\rho} \cdot \rho(x) = [e \cdots x^\rho] = \square [e'^{\rho, \sigma_\theta}]$ by a β -step.

Rule (2) (pattern split): an assignment ρ' of the refined left-hand side $L[\widehat{c}_i(\vec{z}_i)/x]$ induces the assignment $\rho = \rho'[x \mapsto [c_i(\vec{e})]]$ of L ; then

$$L[\widehat{c}_i(\vec{z}_i)/x]^{\mathfrak{M},\rho'} = L^{\mathfrak{M},\rho} = [\text{case}(x; \dots)^{\rho, \sigma_\theta}],$$

and a head ι -step reduces the representative to $[(b_i[c_i(\vec{z}_i)/x])^{\rho', \sigma_\theta}]$.

Rule (3) (flatten): the emitted axiom $L \approx g(\mathcal{C}_\theta(s), \vec{w}')$ is valid: by the invariant and Lemma 5.9, both sides denote $[\text{case}(s; \vec{b}r)^{\rho, \sigma_\theta}]$ (for the right side, $g^{\mathfrak{M}}$ applies its witness $\lambda z.\text{case}(z; \vec{b}r)$ to $\mathcal{C}_\theta(s)^{\mathfrak{M},\rho} = [s^{\rho, \sigma_\theta}]$ and one β -step concludes). The recursive call's invariant is the root computation for case lifts above.

Rule (4) (inner fix): the emitted axiom $L \approx f^0(\vec{w})$ is valid since both sides denote $[e^{\rho, \sigma_\theta}]$ (the right side by the definition of $(f^0)^{\mathfrak{M}}$ with witness e); the recursive compilation is covered by the fix-lift root case.

Rule (5): the emitted axiom $L \approx \mathcal{C}_\theta(e)$ is valid by the invariant and Lemma 5.9.

Pointer axioms: $\forall \vec{z}. \widehat{c}^0 @ z_1 @ \dots @ z_{a_c} \approx \widehat{c}(\vec{z})$ holds because both sides denote $[c \vec{e}]$; similarly for fix-lift pointers. $\square$

5.5 Coincidence: the translation reflects the semantics

Lemma 5.11 (Coincidence). *Let σ be a shallow set type and φ a shallow proposition (over $\mathbb{E}$ and a context of term variables), and let ρ be a valuation of their free variables. Then:*

1. *for every first-order term u over the variables of ρ : $\mathfrak{M}, \rho \models \mathcal{G}_\sigma(u) \iff u^{\mathfrak{M},\rho} \in \llbracket \sigma \rrbracket_\rho$;*
2. *$\mathfrak{M}, \rho \models \mathcal{F}(\varphi) \iff \langle \varphi \rangle_\rho = 1$.*

Proof. Mutual induction on the structure of σ and φ , following Definition 4.7.

- $\sigma = I$: $\mathcal{G}_I(u) = \widehat{I}(u)$ and $\widehat{I}^{\mathfrak{M}} = \llbracket I \rrbracket$ by Definition 5.7.
- $\sigma = \Pi x:\sigma'.\tau$: $\mathfrak{M}, \rho \models \forall x. \mathcal{G}_{\sigma'}(x) \rightarrow \mathcal{G}_{\tau}(u @ x)$ iff for all $d \in \mathcal{D}$ with $d \in \llbracket \sigma' \rrbracket_{\rho}$ (induction hypothesis (1) at the variable x) we have $u^{\mathfrak{M}, \rho} \cdot d \in \llbracket \tau \rrbracket_{\rho[x \mapsto d]}$ (induction hypothesis (1) for τ , at the assignment $\rho[x \mapsto d]$ and the term $u @ x$), which is the definition of $u^{\mathfrak{M}, \rho} \in \llbracket \Pi x:\sigma'.\tau \rrbracket_{\rho}$.
- $\sigma = \varphi' \rightarrow \tau$ and $\sigma = \{x:\sigma' \mid \varphi'\}$: as above, using induction hypothesis (2) for φ' ; the refinement case uses the standard first-order substitution lemma, $\mathfrak{M}, \rho \models \mathcal{F}(\varphi')[u/x] \iff \mathfrak{M}, \rho[x \mapsto u^{\mathfrak{M}, \rho}] \models \mathcal{F}(\varphi')$.
- Connectives, $\top$, $\perp$: the classical satisfaction clauses match the truth-value clauses of Definition 5.3 literally.
- $\mathcal{F}(\forall x:\sigma'.\varphi') = \forall x. \mathcal{G}_{\sigma'}(x) \rightarrow \mathcal{F}(\varphi')$: satisfaction quantifies over all $d \in \mathcal{D}$ and relativises by the guard, which by induction hypothesis (1) carves out exactly $\llbracket \sigma' \rrbracket_{\rho}$; this matches $\min_{d \in \llbracket \sigma' \rrbracket_{\rho}}$. Dually for $\exists$.
- Equations: $\mathfrak{M}, \rho \models \mathcal{C}(\mathcal{E}(a)) \approx \mathcal{C}(\mathcal{E}(b))$ iff $\mathcal{C}(\mathcal{E}(a))^{\mathfrak{M}, \rho} = \mathcal{C}(\mathcal{E}(b))^{\mathfrak{M}, \rho}$ iff $[\mathcal{E}(a)^{\rho}] = [\mathcal{E}(b)^{\rho}]$ (by Lemma 5.9 with $\theta = \emptyset$) iff $\llbracket a \rrbracket_{\rho} = \llbracket b \rrbracket_{\rho}$, which is $\langle a =_{\sigma} b \rangle_{\rho} = 1$.
- J -atoms: as for equations, using $\widehat{J}^{\mathfrak{M}} = \langle J \rangle$.

Shallowiness is used exactly here: predicate quantifiers and predicate-variable atoms have no translation, and all remaining formers are covered. $\square$

6 Soundness

This section proves the main results: every axiom of $\Delta(\mathbb{E}, G)$ holds in $\mathfrak{M}$, and consequently first-order provability of $\mathcal{F}(G)$ entails truth of G in the semantics of Section 5.

6.1 Satisfaction of contexts and guard disciplines

Definition 6.1 (Context satisfaction). Let Γ be a context. A valuation ρ *satisfies* Γ , written $\rho \models \Gamma$, if for every declaration, in telescope order: $\rho(x) \in \llbracket \sigma \rrbracket_{\rho}$ for $x : \sigma$; $\rho(q) : \mathcal{D}^n \rightarrow \{0, 1\}$ for $q : (\sigma_1, \dots, \sigma_n) \rightarrow \text{Prop}$; and $\langle \varphi \rangle_{\rho} = 1$ for $p : \varphi$.

The treatment of `fix` requires interpolating between “ $\rho(f)$ is anything” and “ $\rho(f)$ inhabits its type”: a rank-restricted membership, in the spirit of type-based termination [BFG⁺04, Gim95].

Lemma 6.2 (Flattening of telescopes). *Let $T = \Pi x_1:\sigma_1 \dots \Pi x_n:\sigma_n.\tau$. Then $F \in \llbracket T \rrbracket_\rho$ iff for all $d_1, \dots, d_n$ chosen successively with*

$$d_i \in \llbracket \sigma_i \rrbracket_{\rho[x_1 \mapsto d_1, \dots, x_{i-1} \mapsto d_{i-1}]}$$

we have $F \cdot d_1 \dots d_n \in \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]}$.

Proof. Immediate by unfolding Definition 5.3 n times: membership in a Π -interpretation constrains only the results of application. $\square$

Definition 6.3 (Rank-restricted interpretation). Let $T = \Pi x_1:\sigma_1 \dots \Pi x_n:\sigma_n.\tau$ be a fix type with guard position k ($\sigma_k = I$ a datatype) and let $m \in \mathbb{N}$. Define $\llbracket T \rrbracket_\rho^{k, < m}$ as the set of $F \in \mathcal{D}$ such that for all successively chosen $d_1, \dots, d_n$ as in Lemma 6.2 with additionally $\text{rk}_I(d_k) < m$:

$$F \cdot d_1 \dots d_n \in \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]}.$$

By Lemma 6.2 and finiteness of ranks (Lemma 5.4), $\llbracket T \rrbracket_\rho = \bigcap_{m \in \mathbb{N}} \llbracket T \rrbracket_\rho^{k, < m}$, and $\llbracket T \rrbracket_\rho^{k, < m}$ is antitone in m .

Definition 6.4 (Guard discipline). A *guard discipline* $\mathcal{D}$ over a context Γ is a finite set of triples (f, V_f, m_f) where the f are pairwise distinct term variables declared in Γ with fix types T_f (guard position k_f , guard datatype I_f), V_f is a set of term variables declared in Γ with type I_f , and $m_f \in \mathbb{N}$. A phrase t (term, proof, type, proposition or predicate expression) *respects* $\mathcal{D}$ if for each $(f, V_f, m_f) \in \mathcal{D}$, every occurrence of f in t lies at the head of an application spine $f a_1 \dots a_{n_f}$ occurring as a subterm of t , with $a_{k_f} \in V_f$. A valuation ρ satisfies Γ under $\mathcal{D}$, written $\rho \models_{\mathcal{D}} \Gamma$, if ρ satisfies all declarations of Γ as in Definition 6.1 except that for each $(f, V_f, m_f) \in \mathcal{D}$ only $\rho(f) \in \llbracket T_f \rrbracket_\rho^{k_f, < m_f}$ is required, and additionally $\text{rk}_{I_f}(\rho(y)) < m_f$ for every $y \in V_f$.

Lemma 6.5 (Locality of respect). *Let t respect $\mathcal{D}$ and let s be a subterm of t occurring in a non-applied position (an argument of an application or constructor, a branch or scrutinee of a case, the body of a binder, a component of any non-application former — that is, any position other than the function part of an application). Then s respects $\mathcal{D}$.*

Proof. Let f occur in s and let $f a_1 \dots a_{n_f}$ be the witnessing spine in t . The spine is the chain of application nodes above this occurrence of f ; if it were not entirely contained in s , then s itself would be a proper prefix $f a_1 \dots a_l$ ($l < n_f$) of the spine and would occur applied (as the function part of the next application node), contradicting the assumption on the position of s . Hence the spine, and the condition $a_{k_f} \in V_f$, witness respect in s . $\square$

6.2 The fundamental lemma

Throughout this subsection we assume the following *environment hypothesis* for the fixed environment $\mathbb{E}$ (discharged in Theorem 6.8):

- (H) for every term definition $c := t : \tau_c$ in $\mathbb{E}$, $[c] \in \llbracket \tau_c \rrbracket$; and for every lemma $c := \pi : \varphi_c$ in $\mathbb{E}$, $\langle \varphi_c \rangle = 1$.

Lemma 6.6 (Fundamental lemma). *Assume (H). Let $\mathcal{D}$ be a guard discipline over Γ and $\rho \models_{\mathcal{D}} \Gamma$.*

1. *If $\Gamma \vdash t : \sigma$ and t respects $\mathcal{D}$, then $\llbracket t \rrbracket_{\rho} \in \llbracket \sigma \rrbracket_{\rho}$.*
2. *If $\Gamma \vdash \pi : \varphi$ and π respects $\mathcal{D}$, then $\langle \varphi \rangle_{\rho} = 1$.*

Proof. By induction on the size of the typing derivation, mutually for (1) and (2). We freely use Lemma 5.4 (weakening), Lemma 5.5 (semantic substitution) and Lemma 5.6 (conversion invariance, which handles the two conversion rules at once), and Lemma 6.5 to propagate the discipline to sub-phrases; types, propositions and predicate expressions contain no applied occurrences of disciplined variables in well-formed judgements (they may mention f only inside embedded terms, where the same locality argument applies). When we extend ρ at bound variables we implicitly α -rename so that disciplined variables and V_f -variables are not shadowed.

Variables. If $t = x$ is not disciplined, $\rho(x) \in \llbracket \sigma \rrbracket_{\rho}$ by $\rho \models_{\mathcal{D}} \Gamma$. A bare occurrence of a disciplined f does not respect $\mathcal{D}$, so this case cannot arise for it.

Constants. For a term constant, $\llbracket c \rrbracket_{\rho} = [c] \in \llbracket \tau_c \rrbracket = \llbracket \tau_c \rrbracket_{\rho}$ by (H) and weakening (types of definitions are closed); the conversion rule may have changed the displayed type, which Lemma 5.6 absorbs. For a lemma constant in (2), directly by (H).

Disciplined spines. Suppose $t = f a_1 \cdots a_j$ with $(f, V_f, m_f) \in \mathcal{D}$ and $j \geq n_f$ (respect makes $f a_1 \cdots a_{n_f}$ a spine of t , and t itself extends it), and let

$$T_f = \Pi x_1 : \sigma_1 \dots \Pi x_{n_f} : \sigma_{n_f} . \tau.$$

The typing derivation decomposes into j applications; by induction hypothesis (1) applied to each argument (which respects $\mathcal{D}$ by Lemma 6.5), $d_i := \llbracket a_i \rrbracket_{\rho} \in \llbracket \sigma_i \rrbracket_{\rho[\dots]}$ for $i \leq n_f$, where the intermediate assignments are handled by semantic substitution exactly as in the ordinary application case below. By respect, $a_{k_f} \in V_f$, so $d_{k_f} = \rho(a_{k_f})$ has $\text{rk}_{I_f}(d_{k_f}) < m_f$; since $\rho(f) \in \llbracket T_f \rrbracket_{\rho}^{k_f, < m_f}$,

$$\llbracket f a_1 \cdots a_{n_f} \rrbracket_{\rho} = \rho(f) \cdot d_1 \cdots d_{n_f} \in \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]} = \llbracket \tau[\vec{a}/\vec{x}] \rrbracket_{\rho},$$

the last step by semantic substitution. Any remaining arguments $a_{n_f+1}, \dots, a_j$ are then absorbed by the ordinary application case.

Application $t a$ (head not disciplined). By induction hypothesis, $\llbracket t \rrbracket_\rho \in \llbracket \Pi x:\sigma.\tau \rrbracket_\rho$ (the function part respects $\mathcal{D}$: its disciplined occurrences have their spines within it, by the argument of Lemma 6.5 applied to the maximal spine decomposition) and $\llbracket a \rrbracket_\rho \in \llbracket \sigma \rrbracket_\rho$. Then $\llbracket t a \rrbracket_\rho = \llbracket t \rrbracket_\rho \cdot \llbracket a \rrbracket_\rho \in \llbracket \tau \rrbracket_{\rho[x \mapsto \llbracket a \rrbracket_\rho]} = \llbracket \tau[a/x] \rrbracket_\rho$.

Application $t \pi$. $\mathcal{E}(t \pi) = \mathcal{E}(t)$. By induction hypothesis (2), $\langle \varphi \rangle_\rho = 1$, so by induction hypothesis (1) and the clause for $\llbracket \varphi \rightarrow \tau \rrbracket_\rho$, $\llbracket t \rrbracket_\rho \in \llbracket \tau \rrbracket_\rho$.

Abstractions. For $\lambda x:\sigma.t$ and $e \in \llbracket \sigma \rrbracket_\rho$: $\llbracket \lambda x:\sigma.t \rrbracket_\rho \cdot e = [(\lambda x.\mathcal{E}(t)^\rho) \hat{e}] = [\mathcal{E}(t)^\rho[x \mapsto e]]$ (one β -step, $\hat{e}$ a representative) $= \llbracket t \rrbracket_{\rho[x \mapsto e]} \in \llbracket \tau \rrbracket_{\rho[x \mapsto e]}$ by induction hypothesis. For $\lambda p:\varphi.t$: if $\langle \varphi \rangle_\rho = 1$ then $\rho \models_{\mathcal{D}} \Gamma, p:\varphi$ and $\llbracket \lambda p:\varphi.t \rrbracket_\rho = \llbracket t \rrbracket_\rho \in \llbracket \tau \rrbracket_\rho$ by induction hypothesis, as required by the clause for $\varphi \rightarrow \tau$.

Constructors. For $c(\vec{a})$: by induction hypothesis each $\llbracket a_j \rrbracket_\rho$ lies in the interpretation of its argument type; recursive arguments lie in $\llbracket I \rrbracket$ with finite ranks (Lemma 5.4), so $\llbracket c(\vec{a}) \rrbracket_\rho = [c(\dots)] \in \llbracket I \rrbracket^{m+1} \subseteq \llbracket I \rrbracket$ for m the maximum of their ranks.

Case. Let $t = \text{case}(a; z.\tau; \{c_i(\vec{y}_i) \Rightarrow b_i\}_i)$ with $a : I$. By induction hypothesis $\llbracket a \rrbracket_\rho \in \llbracket I \rrbracket$, so $\llbracket a \rrbracket_\rho = [c_i(\vec{e})]$ for some i and representatives with $[e_j] \in \llbracket \sigma_{ij} \rrbracket$ (non-recursive) resp. $\llbracket I \rrbracket$ (recursive), by Definition 5.3. Let $\rho' = \rho[\vec{y}_i \mapsto [\vec{e}]]$. Then

$$\mathcal{E}(t)^\rho = \text{case}(\mathcal{E}(a)^\rho; \dots) =_{\square} \text{case}(c_i(\vec{e}); \dots) \rightarrow_{\square} \mathcal{E}(b_i)^{\rho'},$$

so $\llbracket t \rrbracket_\rho = \llbracket b_i \rrbracket_{\rho'}$, and $\rho' \models_{\mathcal{D}} \Gamma, \vec{y}_i:\vec{\sigma}_i$. By induction hypothesis, $\llbracket b_i \rrbracket_{\rho'} \in \llbracket \tau[c_i(\vec{y}_i)/z] \rrbracket_{\rho'} = \llbracket \tau \rrbracket_{\rho[z \mapsto \llbracket a \rrbracket_\rho]} = \llbracket \tau[a/z] \rrbracket_\rho$, using semantic substitution twice and $\llbracket c_i(\vec{y}_i) \rrbracket_{\rho'} = \llbracket a \rrbracket_\rho$.

Fix. Let

$$t = \text{fix } f:T. \lambda \vec{x}. \text{case}(x_k; \dots; \{c_i(\vec{y}_i) \Rightarrow b_i\}_i)$$

with $T = \Pi x_1:\sigma_1 \dots \Pi x_n:\sigma_n.\tau$ and $\sigma_k = I$, and let $F := \llbracket t \rrbracket_\rho$. Since $\llbracket T \rrbracket_\rho = \bigcap_m \llbracket T \rrbracket_\rho^{k, < m}$, it suffices to prove $F \in \llbracket T \rrbracket_\rho^{k, < m}$ for all m , by strong induction on m (the *rank induction*, nested inside the main induction).

So let m be given, with $F \in \llbracket T \rrbracket_\rho^{k, < m'}$ for all $m' < m$, and let $d_1, \dots, d_n$ be a chain as in Definition 6.3 with $m_0 := \text{rk}_I(d_k) < m$. Then $d_k = [c_i(\vec{e})]$ for some i , with recursive components of rank $< m_0$ and non-recursive components in their interpretations. Unfolding one μ -step, n β -steps and one ι -step on a representative of $F \cdot d_1 \dots d_n$ (as in the case case above):

$$F \cdot d_1 \dots d_n = \llbracket b_i \rrbracket_{\rho'}, \quad \rho' := \rho[f \mapsto F, \vec{x} \mapsto \vec{d}, \vec{y}_i \mapsto [\vec{e}]].$$

Here the substituted occurrence of the fix term for f represents the class F , so the equation reads $\llbracket b_i \rrbracket$ at the displayed valuation. Now apply the *main* induction hypothesis to the (structurally smaller) derivation of

$$\Gamma, f:T, \vec{x}:\vec{\sigma}, \vec{y}_i:\vec{\sigma}'_i \vdash b_i : \tau[c_i(\vec{y}_i)/x_k],$$

with discipline $\mathcal{D}' = \mathcal{D} \cup \{(f, \vec{y}_i^{\text{ec}}, m_0)\}$ and valuation ρ' . The premises hold: b_i respects $\mathcal{D}'$ (for f by the guard condition grd_k of the fix rule; for the outer disciplines by Lemma 6.5); $\rho'(f) = F \in \llbracket T \rrbracket_\rho^{k, < m_0}$ by the rank induction hypothesis ($m_0 < m$); $\text{rk}_I(\rho'(y)) < m_0$ for $y \in \vec{y}_i^{\text{ec}}$ as noted; and the remaining declarations are satisfied outright. The conclusion $\llbracket b_i \rrbracket_{\rho'} \in \llbracket \tau[c_i(\vec{y}_i)/x_k] \rrbracket_{\rho'} = \llbracket \tau \rrbracket_{\rho[\vec{x} \mapsto \vec{d}]}$ follows by semantic substitution as in the case case. This closes the rank induction, hence the fix case.

Refinement formers. $\text{refine}(t, \pi)$: by induction hypotheses, $\llbracket t \rrbracket_\rho \in \llbracket \sigma \rrbracket_\rho$ and $\langle \varphi[t/x] \rangle_\rho = 1$, i.e. $\langle \varphi \rangle_{\rho[x \mapsto \llbracket t \rrbracket_\rho]} = 1$ by semantic substitution; since $\mathcal{E}(\text{refine}(t, \pi)) = \mathcal{E}(t)$, this is membership in $\llbracket \{x:\sigma \mid \varphi\} \rrbracket_\rho$. $\text{val}(t)$: membership in the refinement interpretation includes membership in $\llbracket \sigma \rrbracket_\rho$, and $\mathcal{E}(\text{val}(t)) = \mathcal{E}(t)$. For $\text{valid}(t)$ in (2): the payload component of the same membership, rewritten by semantic substitution to $\langle \varphi[\text{val}(t)/x] \rangle_\rho = 1$ using $\llbracket \text{val}(t) \rrbracket_\rho = \llbracket t \rrbracket_\rho$.

Singleton eliminations. $\text{split}(\pi; p_1 p_2.t)$: by induction hypothesis (2), $\langle \varphi_1 \wedge \varphi_2 \rangle_\rho = 1$, so both conjuncts hold and $\rho \models_{\mathcal{D}} \Gamma, p_1:\varphi_1, p_2:\varphi_2$; conclude by induction hypothesis (1) on t , since the erasures agree. $\text{cast}(\pi; z.\tau; t)$: induction hypothesis (2) gives $\llbracket a \rrbracket_\rho = \llbracket b \rrbracket_\rho$, hence $\llbracket \tau[a/z] \rrbracket_\rho = \llbracket \tau \rrbracket_{\rho[z \mapsto \llbracket a \rrbracket_\rho]} = \llbracket \tau[b/z] \rrbracket_\rho$; conclude by induction hypothesis on t . $\text{efq}(\pi; \sigma)$ (and $\text{efq}(\pi; \varphi)$ in (2)): induction hypothesis (2) yields $\langle \perp \rangle_\rho = 1$, a contradiction, so the case is vacuous: no valuation satisfying the context reaches it.

Proof formers (2). The rules for $\rightarrow, \wedge, \vee, \forall, \exists, \top$ and predicate quantification are sound for the truth-value semantics of Definition 5.3 by direct computation — each introduction/elimination clause matches the corresponding min/max clause; instantiation rules use semantic substitution (parts (1) and (3) of Lemma 5.5), and the witnesses in ecase and $\forall q$ -elimination are semantic (an element $d \in \llbracket \sigma \rrbracket_\rho$, a predicate $\langle P \rangle_\rho$), requiring no syntactic counterpart. refl is reflexivity of class equality; rew uses $\llbracket a \rrbracket_\rho = \llbracket b \rrbracket_\rho$ and semantic substitution exactly as $\text{cast}(\cdot; \cdot)$.

Predicate-constructor introduction. For $k(\vec{a}; \vec{\pi})$: by induction hypotheses the telescope values lie in their interpretations, non-recursive premises are true, and each recursive premise $J(\vec{v})$ yields a tuple in $\langle J \rangle$, hence in some finite stage; the definition of $\langle J \rangle^{m+1}$ (at m the maximal such stage) puts the conclusion tuple in $\langle J \rangle$, and semantic substitution identifies it with $(\llbracket u_l[\vec{a}/\vec{x}] \rrbracket_\rho)_l$.

Induction (ind_J). Let $\pi_0 : J(\vec{a})$ and let $\vec{\pi}$ be the clause proofs. By induction hypothesis (2), $(\llbracket a_l \rrbracket_\rho)_l \in \langle J \rangle$, and for each clause j , $\langle \Xi_j(\vec{z}.\psi) \rangle_\rho = 1$. We show by strong induction on r that every tuple $\vec{d} \in \langle J \rangle^r$ satisfies $\langle \psi \rangle_{\rho[\vec{z} \mapsto \vec{d}]} = 1$: a tuple enters $\langle J \rangle^r$ ($r = r' + 1$) through some clause j with witnesses $\vec{d}' \in \llbracket \vec{\tau}_j \rrbracket$, true non-recursive premises, and recursive-premise tuples in $\langle J \rangle^{r'}$; instantiating the (semantic) universal quantifiers and implications of $\langle \Xi_j \rangle_\rho = 1$ at these witnesses — with the induction-hypothesis premises supplied by the inner induction at r' — yields $\langle \psi[\vec{u}_j/\vec{z}] \rangle_{\rho[\vec{x} \mapsto \vec{d}]} = 1$, which

by semantic substitution is $\langle \psi \rangle_{\rho[\vec{z} \mapsto \vec{d}]} = 1$. Conclude at the tuple $(\llbracket a_l \rrbracket_\rho)_l$. The datatype induction ind_I is identical with ranks in place of stages. $\square$

6.3 Validity of the emitted theory

Lemma 6.7 (Validity of specification axioms). *Let $c := t : \tau$ be a term definition with $\tau \in \mathfrak{S}$, and suppose $[c] \in \llbracket \tau \rrbracket$. Then $\mathfrak{M} \models \mathcal{S}(c)$.*

Proof. We prove the following invariant of the recursion in Definition 4.8, by induction on τ' : if ρ assigns the variables accumulated so far, $v := [c \hat{e}_1 \cdots \hat{e}_j]$ where $\hat{e}_i$ are representatives of the accumulated informative-argument values, and $v \in \llbracket \tau' \rrbracket_\rho$, then $\mathfrak{M}, \rho \models S(u; \tau')$. First observe that whichever rendering the accumulator receives — $\hat{c}(\vec{x})$ when saturated, the pointer chain $\hat{c}^0 @ x_1 @ \cdots @ x_j$ when not, with @-appended extras — its denotation under ρ is exactly v , by Definition 5.7.

If $\tau' = \Pi x : \sigma. \tau''$: for d with $\mathfrak{M}, \rho \models \mathcal{G}_\sigma(x)[x \mapsto d]$, Lemma 5.11 gives $d \in \llbracket \sigma \rrbracket_\rho$, hence $v \cdot d \in \llbracket \tau'' \rrbracket_{\rho[x \mapsto d]}$, and $v \cdot d$ is the denotation of the extended accumulator; conclude by the induction hypothesis. If $\tau' = \varphi \rightarrow \tau''$: assuming $\mathfrak{M}, \rho \models \mathcal{F}(\varphi)$, Lemma 5.11 gives $\langle \varphi \rangle_\rho = 1$, hence $v \in \llbracket \tau'' \rrbracket_\rho$, and the accumulator is unchanged — matching the unchanged arity of the erased program. If τ' is a datatype or refinement type: $\mathfrak{M}, \rho \models \mathcal{G}_{\tau'}(\bar{u})$ by Lemma 5.11 applied to $v \in \llbracket \tau' \rrbracket_\rho$.

The lemma is the invariant at the root ($j = 0$, $v = [c]$, ρ empty). $\square$

Theorem 6.8 (Environment validity). *Let $\mathbb{E}$ be a well-formed definitional environment with $\mathfrak{S}$ declarations, and G a closed shallow goal. Then:*

1. (H) holds: $[c] \in \llbracket \tau_c \rrbracket$ for every term definition and $\langle \varphi_c \rangle = 1$ for every lemma of $\mathbb{E}$;
2. $\mathfrak{M} \models \Delta(\mathbb{E}, G)$.

Proof. (1) By induction on the position of the definition in $\mathbb{E}$. For $c := t : \tau_c$: the body is typed over the strict prefix, for which (H) holds by induction; Lemma 6.6 with empty context, empty discipline and empty valuation gives $\llbracket t \rrbracket \in \llbracket \tau_c \rrbracket$, and $[c] = [\mathcal{E}(t)] = \llbracket t \rrbracket$ by the δ -rule of $\lambda_\square$. For a lemma, identically via part (2) of the fundamental lemma.

(2) We check each axiom family of Definition 4.10.

Compilation, pointer and goal-auxiliary axioms: this is Lemma 5.10.

Specification axioms: Lemma 6.7, using (1).

Premise axioms: (1) gives $\langle \varphi_c \rangle = 1$; Lemma 5.11 (closed formula, empty valuation) gives $\mathfrak{M} \models \mathcal{F}(\varphi_c)$.

Constructor typing: given $\vec{d}$ with $\mathfrak{M} \models \mathcal{G}_{\sigma_{ij}}(y_j)[\vec{y} \mapsto \vec{d}]$, Lemma 5.11 places each d_j in $\llbracket \sigma_{ij} \rrbracket$; recursive ones have finite rank, so $\hat{c}_i^{\mathfrak{M}}(\vec{d}) \in \llbracket I \rrbracket^{m+1} \subseteq \llbracket I \rrbracket = \hat{I}^{\mathfrak{M}}$.

Inversion: if $d \in \widehat{I}^{\mathfrak{M}} = \llbracket I \rrbracket$ then $d \in \llbracket I \rrbracket^{m+1}$ for some m , so $d = [c_i(\vec{e})]$ with component classes in the required interpretations; instantiate the existential quantifiers of the i -th disjunct with these classes. The guards hold by Lemma 5.11 and the equation $d \approx \widehat{c}_i(\vec{y})$ by the interpretation of $\widehat{c}_i$.

Injectivity and discrimination: Corollary 3.4(1); note that neither needs any guard — they hold at all elements of $\mathcal{D}$.

Predicate clause axioms: as constructor typing, using the stage construction of $\langle J \rangle$, Lemma 5.11 for the premises, and Lemma 5.9 to identify $\mathcal{C}(\mathcal{E}(u_i))^{\mathfrak{M}, \rho}$ with $\llbracket u_i \rrbracket_\rho$.

Predicate case-analysis axioms: as inversion, unfolding the stage that admitted the tuple: its witnesses instantiate the existentials, the guard and premise conjuncts hold by Lemma 5.11 (recursive premises: their tuples are in $\langle J \rangle$, i.e. the corresponding $\mathcal{F}()$ -atoms hold), and the index equations by Lemma 5.9. $\square$

6.4 Main theorem and corollaries

Theorem 6.9 (Soundness). *Let $\mathbb{E}$ be a well-formed definitional environment with $\mathfrak{S}$ declarations and G a closed shallow proposition over $\mathbb{E}$. If*

$$\Delta(\mathbb{E}, G) \vdash_{\text{FOL}} \mathcal{F}(G)$$

then $\langle G \rangle = 1$, i.e. G is true in the proof-irrelevant classical semantics of $\mathbb{E}$.

Proof. By Theorem 6.8, $\mathfrak{M} \models \Delta(\mathbb{E}, G)$. By soundness of classical first-order logic [Sho67], $\mathfrak{M} \models \mathcal{F}(G)$. By Lemma 5.11 at the empty valuation, $\langle G \rangle = 1$. $\square$

Corollary 6.10 (Consistency). *$\Delta(\mathbb{E}, G)$ is consistent, for every well-formed definitional $\mathbb{E}$ and goal G .*

Proof. $\mathfrak{M} \models \Delta(\mathbb{E}, G)$ and $\mathfrak{M} \not\models \perp$. Note that no consistency assumption on the source is needed: a definitional environment always has the model $\mathfrak{M}$. $\square$

Corollary 6.11 (Refutable goals are not provable). *If $\vdash \pi : \neg G$ for some closed proof π over $\mathbb{E}$, then $\Delta(\mathbb{E}, G) \not\vdash_{\text{FOL}} \mathcal{F}(G)$. More generally, $\Delta(\mathbb{E}, G)$ never proves both $\mathcal{F}(G)$ and $\mathcal{F}(\neg G)$.*

Proof. By Lemma 6.6 (via Theorem 6.8(1)), $\langle \neg G \rangle = 1$, so $\langle G \rangle = 0$ and Theorem 6.9 applies contrapositively. For the second statement: $\mathcal{F}(\neg G) = \mathcal{F}(G) \rightarrow \perp$, and $\mathfrak{M}$ satisfies every consequence of $\Delta(\mathbb{E}, G)$. $\square$

Proposition 6.12 (Standardness on first-order datatypes). *Call a datatype first-order if all constructor argument types are (recursively) first-order datatypes. For such I , the map assigning to each closed constructor term its class is a bijection between the free term algebra of I 's constructors and $\llbracket I \rrbracket$.*

In particular $n \mapsto [\mathbb{S}^n(\mathbf{O})]$ is a bijection $\mathbb{N} \rightarrow \llbracket \mathbf{nat} \rrbracket$, and for propositions whose quantifiers all range over first-order datatypes, $\langle \cdot \rangle$ is satisfaction in the corresponding standard structure (with each constant c interpreted by the function $\vec{d} \mapsto [c \vec{e}]$, which maps datatype interpretations to datatype interpretations by Lemma 6.6).

Proof. Injectivity: Corollary 3.4(1), by induction on constructor terms. Surjectivity: by induction on stages, every element of $\llbracket I \rrbracket^m$ is the class of a closed constructor term whose arguments are, inductively, classes of constructor terms. The last claim unfolds Definition 5.3: on such propositions the clauses of $\langle \cdot \rangle$ quantify exactly over the sets $\llbracket I \rrbracket$ and interpret atoms by class equality and the lfp relations, which under the bijection is the standard Tarskian satisfaction. $\square$

Remark 6.13 (Postulated axioms). If $\mathbb{E}$ additionally contains postulates $c : \varphi$ without bodies, all results relativise: Theorem 6.8 goes through provided $\langle \varphi \rangle = 1$ for each postulate — the theorems then hold for every choice of semantics validating the postulates. For postulates true in the term-model semantics (excluded middle and proof irrelevance are; note that functional extensionality is *not* — see Section 10.5) nothing changes; an inconsistent postulate set naturally voids the guarantee.

Remark 6.14 (What is *not* claimed). Theorem 6.9 does not claim that G is *provable* in CIC^- : the semantics validates classical logic and proof irrelevance, so e.g. $\mathcal{F}(G \vee \neg G)$ is first-order provable while $G \vee \neg G$ need not be CIC^- -provable. This matches the design of the implemented tool, which runs classical provers and relies on reconstruction; see Section 1.3 and Section 10.7. The proof-theoretic sharpening — recovering a CIC^- proof from a first-order derivation, in the style of [Cza18] — is future work.

7 Per-constructor equations versus guarded disjunctive case axioms

The translation of Section 4 renders a pattern-matching definition as per-constructor equations. The translation of [CK18, §5.2] renders the same definition as a single *guarded disjunctive axiom*. This section compares the two forms inside the framework just built: the model $\mathfrak{M}$ arbitrates which guards are semantically necessary, and first-order derivations settle the logical relationship. Fix a definition by a case split on a variable,

$$c := \lambda x:I. \text{case}(x; z.\tau; \{c_i(\vec{y}_i) \Rightarrow b_i\}_i) : \Pi x:I. \tau,$$

and abbreviate $R_i := \mathcal{C}(\mathcal{E}(b_i))$ (over the variables $\vec{y}_i$). Our translation emits the *split form*

$$\forall \vec{y}_i. \widehat{c}(\widehat{c}_i(\vec{y}_i)) \approx R_i \quad (\text{one axiom per constructor } i), \quad (\text{Eq}_c)$$

whereas the disjunctive rendering is

$$\forall x. \widehat{I}(x) \rightarrow \bigvee_i \exists \vec{y}_i. \bigwedge_j \mathcal{G}_{\sigma_{ij}}(y_{ij}) \wedge x \approx \widehat{c}_i(\vec{y}_i) \wedge \widehat{c}(x) \approx R_i, \quad (\text{Disj}_c)$$

and we write Disj_c^- for the variant with the guard $\widehat{I}(x)$ omitted.

7.1 Semantic status

Proposition 7.1 (The split form needs no guards). $\mathfrak{M} \models \text{Eq}_c$, with the universally quantified variables $\vec{y}_i$ ranging over the entire domain $\mathcal{D}$ — junk included. Moreover $\mathfrak{M} \models \text{Disj}_c$.

Proof. The first claim is an instance of Lemma 5.10: the emitted equations are valid in $\mathfrak{M}$ under *every* assignment; concretely, for arbitrary $\vec{d} \in \mathcal{D}$, $\widehat{c}^{\mathfrak{M}}(\widehat{c}_i^{\mathfrak{M}}(\vec{d})) = [c \ c_i(\vec{e})] =_{\square} [\mathcal{E}(b_i)^{[\vec{y}_i \mapsto \vec{d}]}]$ by a δ -, β - and ι -step — the reduction fires syntactically, with no typing side condition. For the second claim: given $d \in \widehat{I}^{\mathfrak{M}}$, inversion validity (Theorem 6.8) yields a disjunct i and witnesses with $d = \widehat{c}_i^{\mathfrak{M}}(\vec{e})$, and Eq_c supplies $\widehat{c}^{\mathfrak{M}}(d) =$ the corresponding R_i -value. $\square$

Proposition 7.2 (The disjunctive form’s guard is mandatory). Let $\mathbb{E}$ contain nat and the definition mypred of Example 4.6. Then $\mathfrak{M} \not\models \text{Disj}_{\text{mypred}}^-$. Moreover, guard omission in the disjunctive form is not merely false in $\mathfrak{M}$ but absolutely unsound: for a single-constructor datatype U with $\mathbf{u} : ()$ and any definition c_0 by a (trivial) case on U , the theory $\{\text{Disj}_{c_0}^-\} \cup \{\text{discrimination for bool}\}$ is inconsistent.

Proof. For the first claim, take the junk element $d = [\mathbf{O} \ \mathbf{O}] \in \mathcal{D}$ (the constructor constant $\mathbf{O}$ applied to itself). By Corollary 3.4(2), d is not convertible to any constructor form, so both disjuncts of $\text{Disj}_{\text{mypred}}^-$ fail at $x := d$: $d \neq [\mathbf{O}]$ and $d \neq [\mathbf{S}(e)]$ for every e . (The guarded form is unaffected: $d \notin \llbracket \text{nat} \rrbracket = \widehat{\text{nat}}^{\mathfrak{M}}$, again by Corollary 3.4(2) and stage induction.)

For the second claim, $\text{Disj}_{c_0}^-$ has the single disjunct $\forall x. x \approx \widehat{\mathbf{u}} \wedge \dots$, which entails $\forall x y. x \approx y$; instantiating with $\widehat{\text{true}}$ and $\widehat{\text{false}}$ contradicts discrimination. This reproduces, inside our framework, the counterexample that makes the guard on the matched term’s variables mandatory in [CK18, §5.6]. $\square$

The two propositions isolate the semantic content of the design change. The disjunctive axiom asserts *exhaustiveness*: every domain element (in the guarded version, every element satisfying the guard) is of constructor form. Exhaustiveness is a property of the *type*, is false at junk, and therefore requires the guard. The equations assert only *computation*: they are silent off their patterns, hold at every point of the model, and need no guard — neither on the matched term nor on the constructor-argument variables $\vec{y}_i$. In the factored architecture this is no accident: exhaustiveness belongs to specification extraction and is emitted once per datatype (the inversion

axiom, guarded), while the program's equations carry no typing content at all. The guard class that [CK18, §5.6] identifies as impossible to omit is thereby not omitted but *dissolved*.

Remark 7.3 (Guard policy for constructor arguments). In the split form, guards on the fresh constructor-argument variables $\vec{y}_i$ would also be sound (weakening a valid formula), and correspond to the optional closure-variable guards of lambda-lifting (`opt_closure_guards` in the implementation); by Proposition 7.1 they are unnecessary, uniformly with the treatment of lambda-lifted closure variables — but here this is a theorem about $\mathfrak{M}$ rather than a conjecture.

7.2 Logical relationship

Proposition 7.4 (Interderivability relative to structural axioms). *Let Str_I denote the structural axioms of I emitted by Definition 4.10 (constructor typing, inversion, injectivity, discrimination). Then, in classical first-order logic with equality:*

1. $\text{Eq}_c \cup \{\text{inversion}\} \vdash_{\text{FOL}} \text{Disj}_c$;
2. $\{\text{Disj}_c\} \cup \text{Str}_I \vdash_{\text{FOL}}$ the guarded equations $\forall \vec{y}_i. \bigwedge_j \mathcal{G}_{\sigma_{ij}}(y_{ij}) \rightarrow \widehat{c}(\widehat{c}_i(\vec{y}_i)) \approx R_i$ for each i .

The unguarded equations Eq_c are strictly stronger than $\{\text{Disj}_c\} \cup \text{Str}_I$: they are not derivable from it.

Proof. (1) Assume $\widehat{I}(x)$. The inversion axiom yields a disjunct i with witnesses $\vec{y}_i$, their guards, and $x \approx \widehat{c}_i(\vec{y}_i)$; rewriting with this equation in $\widehat{c}(x)$ and applying the i -th equation of Eq_c produces $\widehat{c}(x) \approx R_i$, completing the i -th disjunct of Disj_c .

(2) Fix i and assume the guards of $\vec{y}_i$. By constructor typing, $\widehat{I}(\widehat{c}_i(\vec{y}_i))$; instantiate Disj_c at $x := \widehat{c}_i(\vec{y}_i)$. In the resulting disjunction, every disjunct $i' \neq i$ contains $\widehat{c}_i(\vec{y}_i) \approx \widehat{c}_{i'}(\vec{y}'_{i'})$, refuted by discrimination. The remaining disjunct yields witnesses $\vec{y}'_i$ with $\widehat{c}_i(\vec{y}_i) \approx \widehat{c}_i(\vec{y}'_i)$ and $\widehat{c}(\dots) \approx R_i[\vec{y}'_i]$; injectivity gives $y_{ij} \approx y'_{ij}$, and rewriting $R_i[\vec{y}'_i]$ back along these equations gives the guarded equation.

For strictness, note that all axioms of $\{\text{Disj}_c\} \cup \text{Str}_I$ hold in the modification $\mathfrak{M}'$ of $\mathfrak{M}$ that reinterprets $\widehat{c}$ arbitrarily at arguments outside $\llbracket I \rrbracket$ -constructor forms with junk components — e.g. setting $\widehat{c}^{\mathfrak{M}'}([c_i(\vec{e})]) := [\square]$ whenever some non-recursive $[e_j]$ lies outside its argument-type interpretation — because every axiom of that set constrains $\widehat{c}$ only under guards. The unguarded equation Eq_c fails in $\mathfrak{M}'$ at such a point whenever R_i 's value there differs from $[\square]$, which is the case already for `mypred` (where $R_2 = y$): take $y := [\text{O O}] \neq [\square]$ (Corollary 3.4). $\square$

Thus nothing is lost logically by splitting — the disjunctive content is recoverable with one inversion inference — and something is gained: the unguarded equations are strictly stronger, yet still sound (Proposition 7.1). The remaining difference is operational.

7.3 Clause shapes and prover mechanics

Consider `mypred` (Example 4.6). Clausifying $\text{Disj}_{\text{mypred}}$ requires a Skolem function f for the existential and distribution over the disjunction, yielding six three-literal non-Horn clauses:

$$\begin{array}{ll} \neg \text{nat}(x) \vee x \approx \text{O} \vee x \approx \text{S}(f(x)) & \neg \text{nat}(x) \vee F(x) \approx \text{O} \vee x \approx \text{S}(f(x)) \\ \neg \text{nat}(x) \vee x \approx \text{O} \vee \text{nat}(f(x)) & \neg \text{nat}(x) \vee F(x) \approx \text{O} \vee \text{nat}(f(x)) \\ \neg \text{nat}(x) \vee x \approx \text{O} \vee F(x) \approx f(x) & \neg \text{nat}(x) \vee F(x) \approx \text{O} \vee F(x) \approx f(x) \end{array}$$

(writing F for $\widehat{\text{mypred}}$), with two positive equality literals per clause and the function’s value expressed only through the Skolem term. Nested matches compound this multiplicatively. A superposition prover can use such clauses only through case-splitting search: no clause is an orientable rewrite rule. The split form clausifies to

$$F(\text{O}) \approx \text{O}, \quad F(\text{S}(y)) \approx y :$$

two unit equations, orientable left-to-right under any simplification ordering with the standard precedence heuristics, hence usable by Vampire [KV13] and E [Sch02] as *demodulators* — the prover evaluates `mypred` inside terms at simplification cost, with no search, no Skolem functions and no case splits. For SMT solvers (CVC4/CVC5 [BCD⁺11], Z3 [dMB08]) the equations are ideal trigger material for E-matching, whereas the disjunctive clauses produce case splits per instantiation. For recursive definitions the effect compounds: the Eq-form of `add` (Example 4.6) is exactly the rewrite system a human would write, and equations of this shape are well established as among the most valuable premises in the practice of HOL-based hammers, where definitional packages export precisely the per-constructor `simps` equations [Kra10, BKPU16], and dependently-typed frontends export equational lemmas rather than raw match trees [QCBA25].

The one capability the disjunctive axiom packages more cheaply is a *case split on the scrutinee*: an ATP that has derived $\text{nat}(t)$ obtains “ $t \approx \text{O}$ or $t \approx \text{S}(y)$ ” together with the corresponding value of F in a single clause set. In the split form this requires chaining the (always emitted) inversion axiom with the equations — one extra inference, at the cost of deriving the guard atom. The exchange is measurable empirically; the a priori analysis favours the split form on equational and computational goals and is neutral on inversion-style goals, since inversion axioms are emitted either way.

Remark 7.5 (Reconstruction signal). Since each split axiom is attached to one constructor, an ATP proof that uses it reveals not only which match was

relevant but which *branch* — strictly more information for the reconstruction backend (which maps used axioms back to case analyses) than the single disjunctive axiom’s name.

8 Shallowness

Shallowness is the design constraint that shaped every clause of Section 4; this section makes it precise and verifies it. The operational argument is the one articulated for the core embedding in [Cza18, §1.1]: first-order ATPs are heavily optimised for the primitives of first-order logic — clausification, resolution, superposition with term indexing, E-matching — so a translation is efficient to the extent that the logical structure of the source arrives at the prover *as* first-order structure. A *deep* embedding, in which a source proposition τ becomes a first-order *term* $\mathcal{C}(\tau)$ related to its proofs by an inhabitation predicate $\mathsf{T}(x, \mathcal{C}(\tau))$, forces every logical inference of the object logic to be simulated by instantiations of bridge axioms, and conjectures become existence statements $(\exists y. \mathsf{T}(y, \mathcal{C}(\tau')))$ that require the prover to synthesise a term standing for a proof — both dramatically hostile to first-order proof search [Cza18, §1.1].

Concretely, we verify the following properties of the translation of Section 4.

- S1 (Native logical structure).** $\mathcal{F}$ maps every connective and quantifier of a shallow proposition to the corresponding first-order connective or (guarded) quantifier, and source equality to first-order equality $\approx$, *at translation time* (Definition 4.7). No logical structure of a shallow proposition is reified into a term. Consequence: the clausal form of $\mathcal{F}(\varphi)$ is the clausal form one would write by hand for φ .
- S2 (No inhabitation predicate).** $\Sigma_{\mathbb{E}}$ contains no binary “has-type” predicate and no term encoding of types (Definition 4.1). The only typing-flavoured atoms are the unary guards $\widehat{I}(u)$, occurring exactly at atomic-type leaves of $\mathcal{G}$ — the soft-typing base case, structurally the device used by Mizar’s type predicates [Wie07] and by the type guards of HOL hammers [MP08, BBPS16].
- S3 (Per-occurrence payload expansion).** The propositional content of a refinement type is expanded inline at every occurrence, in both polarities — as a premise under $\forall$ (negative position) and as a conjunct under $\exists$ and in conclusions (positive position) — licensed by the fact that Lemma 5.11 is an equivalence, not an implication. There is no **sig** symbol, no boxing constructor, no projection function and no unfolding axiom in the output: nothing is left for the ATP to unfold. This is the clause that repairs the local relapse into deep embedding identified in

Section 1 — compare the specification axiom of Example 4.9 with the opaque $\text{HasType}(h, \text{sig}(\dots))$ atom it replaces.

S4 (Proofs never become terms). Erasure drops proof arguments and abstractions entirely (Definition 3.5); the specification clause $\mathcal{G}_{\varphi \rightarrow \tau}(u) = \mathcal{F}(\varphi) \rightarrow \mathcal{G}_{\tau}(u)$ keeps the first-order term u unapplied across a propositional premise, in lock-step with the pruned arity of the extracted program. The prover consequently never constructs, matches or instantiates any term standing for a proof. (The dead-code constant $\odot$ can occur, but never as an argument at a pruned position — only as the image of $\perp$ -elimination.)

S5 (Definitions as oriented computation). The definitional content of the environment arrives as per-constructor pattern equations (Definition 4.3) — orientable unit or near-unit equations forming an orthogonal rewrite system (Remark 4.5) — rather than as guarded disjunctions (Section 7.3). Case analysis (a search operation) is segregated into the per-datatype inversion axioms, so provers pay for splitting only when the proof actually requires it.

Two further points delimit what shallowness can and cannot mean here.

The residue. Full first-order structure cannot always be achieved: partial applications go through the application operator $@$ and pointer constants (Definition 4.2) — the standard arity-optimisation residue of every HOL-to-FOL translation [MP08] — and anonymous λ -abstractions, matches and fixpoints inside formulas are lifted to fresh symbols with defining axioms, which is unavoidable in a language without binders. Both mechanisms are local: they affect the translation of *terms*, never the logical skeleton of a proposition. Propositions outside $\mathfrak{S}$ (predicate quantification) have no shallow first-order image at all and are excluded rather than encoded; the implemented tool falls back to opaque atoms for them, a fallback whose soundness is a separate (deep-embedding) question not treated here.

Shallowness versus soundness. The two requirements pull in the same direction in this design. The inline expansion of refinement payloads (S3) is justified by the model equivalence of Lemma 5.11; the guard-free equations (S5) are not merely sound but strictly stronger than their guarded counterparts (Proposition 7.4) while remaining valid in $\mathfrak{M}$ (Proposition 7.1). Where a guard is genuinely load-bearing — exhaustiveness in inversion axioms (Proposition 7.2) — the translation keeps it. The soundness proof thus doubles as a shallowness audit: every remaining non-native atom ($\widehat{I}$ guards, lifted symbols, $@$) is one the model construction shows to be necessary for the statement it protects, and every removable layer of indirection

has been removed at translation time, which is the design maxim shared with the instantiation-level optimisations of the implemented tool: work done at translation time is work the ATP does not repeat at search time, once per relevant inference.

9 Related work

Program extraction. The erasure calculus, the treatment of singleton eliminations, informative-singleton unboxing and logical-argument pruning follow Letouzey [Let04, Let03, Let08], whose extraction descends from Paulin-Mohring’s realizability-based extraction for the Calculus of Constructions [PM89], itself in the tradition of Kleene realizability [Kle45, TvD88]. Our specification extraction $\mathcal{G}$ is precisely the structural realizability predicate of [PM89] and the simulation predicate $\llbracket \cdot \rrbracket$ of [Let04, §2.4], read as a first-order formula about the extracted constant; the fundamental lemma (Lemma 6.6) plays the role of extraction correctness. The differences are dictated by the target: extraction must protect evaluation order (dummy abstractions, argument-absorbing fixpoints), which an equational target does not care about, so erasure can prune outright (Section 3); conversely an equational target cannot rely on lazy non-execution, which is why well-founded recursion is a genuine boundary (Section 10.3) rather than a solved case. Machine-checked accounts of erasure correctness exist in the MetaCoq project [SBF⁺20]; formally connecting our $\mathcal{E}$ to MetaCoq’s verified erasure is an attractive future step.

Hammer translations. The immediate context is the CoqHammer translation [CK18, CK16] and its proof-theoretically sound core, the shallow embedding of proof-irrelevant PTSs into first-order logic [Cza18]. Relative to [Cza18], this paper adds what that paper explicitly leaves out — inductive types, pattern matching, recursion and refinement types — at the price of a model-theoretic rather than proof-theoretic soundness statement, and of assuming a stratified monomorphic fragment rather than an arbitrary logical piPTS. Relative to [CK18], the case-axiom and inhabitation-atom layers are replaced by the extraction-factored design, whose possibility is conjectured in §9 of that paper. For HOL-based hammers the corresponding translation stack — λ -lifting, arity optimisation, type guards and tags with monotonicity-based soundness proofs — is developed by Meng and Paulson [MP08] and Blanchette et al. [BBPS16]; our guard predicates are the unary (“featherweight”) end of that design space, and our model-with-junk construction is the same proof pattern used there for guard-omission soundness. Sledgehammer’s practice [PB12, BKPU16] supplies the operational evidence for S5: HOL hammers export per-constructor `simps` equations produced by the definitional packages [Kra10], never disjunctive match

axioms. Early precedents for encoding dependent products with type predicates in untyped first-order logic go back to Tammet and Smith [TS98]; the types-as-predicates discipline is also Mizar’s [Wie07].

Dependent type theory to ATPs. Lean-auto [QCBA25] translates Lean 4 goals through monomorphisation into higher-order logic, deliberately consuming the elaborator’s *equational lemmas* for defined functions instead of raw match-trees — independent confirmation that equations are the right currency for provers, obtained there from the frontend, here from the translation itself. Šinkarovs and Rawson’s Agda–Vampire bridge [ŠR26] restricts to an equational-Horn fragment shared by both systems, again matching the shape our compilation produces. The Equations package [SM19] generates defining equations and elimination principles inside Coq for its own definitions; our compilation produces the analogous equations uniformly for plain definitions, at the FOL level.

Semantics and termination. The proof-irrelevant, classical interpretation of propositions is the standard proof-irrelevant model construction for CIC-like theories [Wer97, MW03], cut down to a truth-value semantics over a syntactic domain. The rank-restricted interpretations used for the fix case of Lemma 6.6 are the semantic counterpart of guarded recursion in the style of Giménez [Gim95] and of type-based termination [BFG⁺04]; inductive definitions by stages are classical [Acz77]. The confluence proof of Appendix A is Takahashi’s method [Tak95]; orthogonality-based alternatives go back to Klop [Klo80] and are textbook material [BN98].

10 Limitations and future work

The fragment CIC^- was chosen to make every emitted axiom provably sound while covering the phenomena the factored architecture is about: pattern matching, structural recursion, propositional content in types, singleton eliminations. We now delimit the boundary, distinguishing presentational restrictions (liftable with more bookkeeping) from genuine semantic boundaries.

10.1 Polymorphism and monomorphisation

CIC^- has no type variables: every inductive type is declared at a ground instance, and guards are unary per-datatype predicates. This matches the intended pipeline, in which heuristic monomorphisation — instantiating polymorphic definitions and inductive families at the ground instances relevant to a problem, as HOL hammers do [BBPS16] — runs before the translation of this paper. Extending the soundness proof to genuinely polymorphic

environments (type-variable guards, parameterised guard predicates) is orthogonal future work; the design questions there (which type arguments and guards are omissible) are treated in companion notes to this paper and in the monotonicity literature [BBPS16].

10.2 Indexed families and large elimination

Datatypes in CIC^- are non-indexed (inductive *predicates* carry indices, but their proofs are erased). Indexed informative families (vectors, finite sets) require dependent case motives over indices and transport in branches; the semantic machinery (classes, stages, ranks) extends, but erasure of index information interacts with injectivity in ways that need care, and we have not carried it out. Large elimination (types defined by recursion) is excluded outright: it breaks the rigidity of set types (Lemma 2.5) on which the structural definitions of $\mathcal{G}$ and $\llbracket \cdot \rrbracket$ rest. A translation for large-eliminating code would need a different device (e.g. per-instance specialisation).

10.3 Well-founded recursion: a hard constraint

`fix` captures structural recursion only. The extension to well-founded (`Acc`-based) recursion is *not* a routine generalisation, and the obstruction deserves precise statement, since it constrains any implementation of the architecture.

In CIC , a function defined by well-founded recursion — Euclidean division $\text{div} : \forall a b. b \neq 0 \rightarrow \{q \mid \dots\}$ is the canonical example — recurses through a proof of $\text{Acc}(x)$, and its unfolding equation holds *only under the erased propositional preconditions*. Program extraction may erase the precondition entirely: the extracted `div` diverges at $b = 0$, which is harmless because lazy (or guarded) evaluation never forces the offending closure [Let04, Let03, §2.6]. An equational axiomatisation enjoys no such protection: an equation is “already applied everywhere”. Erasing the precondition from the unfolding equation of `div` yields the instance

$$\widehat{\text{div}}(a, 0) \approx S(\widehat{\text{div}}(a, 0)) \quad (\text{for } b = 0, \text{ when the guard test } b \leq a \text{ degenerates}),$$

which contradicts the perfectly legitimate premise $\forall n. n \neq S(n)$ — the resulting theory is *inconsistent*, not merely incomplete, and Corollary 6.10 would fail. The constraint is therefore: *defining equations obtained by recursion through an erased proof must retain the erased propositional premises of the definition as implications* ($\forall a b. \mathcal{F}(b \neq 0) \rightarrow \dots$), the analogue for equations of the evaluation-order problem that forced Letouzey to abandon the erasure rule $\mathcal{E}(\lambda p:\varphi.t) = \mathcal{E}(t)$ for *executable* extraction. In our fragment the unconditional-drop rule of Definition 3.5 is sound precisely because `fix` types contain no propositional premises and unfolding is total on the guard datatype (Lemma 6.6); lifting either restriction requires the premised-equation form, with the semantics interpreting the defined symbol by an arbitrary total extension of the partial function.

10.4 Transport erasure and UIP

The clause $\mathcal{E}(\text{cast}(\pi; z.\tau; t)) = \mathcal{E}(t)$ (and the corresponding validity of unconditional transport equations, e.g. the image of `eq_rect` becoming an identity) is validated by $\mathfrak{M}$ through proof irrelevance: $\langle a = b \rangle = 1$ collapses the classes, so transport is semantically the identity. As a *source* theorem, however, “transport is the identity” at an arbitrary type is uniqueness of identity proofs, which CIC does not prove [GCST19]. The consequence is not unsoundness — the model validates it, and CIC[−]’s `cast( ; ; )` is well typed only at matching endpoints — but a *reconstruction* caveat for the implemented pipeline: an ATP proof essentially using transport-erasure at a type without decidable equality may resist replay inside Coq, the same failure mode the tool already accepts for other semantically-validated principles. A conservative variant guards the transport equation with the source equation as a premise.

10.5 Function extensionality fails in the term model

$\mathfrak{M}$ interprets equality as $\lambda_{\square}$ -convertibility of closed terms, which is intensional: $[\lambda x. \text{add}(\mathbf{O}, x)] \neq [\lambda x. x]$ although the two agree on all of $\llbracket \text{nat} \rrbracket$ (the first is stuck at a variable). So $\langle \text{funext} \rangle = 0$: the guarantee of Theorem 6.9 does not extend to environments postulating function extensionality (Remark 6.13). Repairing this requires a coarser model — quotienting $\mathcal{D}$ by an observational equivalence — which changes the discrimination and compilation arguments and is left as future work. Note the asymmetry: the translation never *emits* an extensionality axiom, so Corollary 6.10 is unaffected; only premise sets containing funext-derived lemmas step outside the present soundness statement.

10.6 Other presentational restrictions

Mutual and nested inductive types, deep (nested-pattern) and mutual recursion, and fixpoints whose recursive calls occur under inner binders with non-variable guard arguments are excluded by the syntactic shape of `fix` and the declarations; all appear liftable by elaborating them into the present forms (mutual recursion via a sum type, deep guards via nested case trees — which the *compilation* already handles, cf. Example 4.6(3) — and course-of-values recursion via auxiliary structural functions), at bookkeeping cost in the `fix` case of Lemma 6.6. Inductive predicates with predicate *parameters* (e.g. `Forall`) are excluded from $\mathfrak{S}$; per-instance specialisation (a fresh predicate symbol per concrete predicate parameter, with clauses inlined) extends the translation shallowly and is the natural implementation route. Enumeration-like datatypes whose constructors carry only propositional data (`sumbool`) are covered as plain datatypes here; inlining their inversion disjunction into

guard positions is a further shallowness optimisation compatible with the model.

10.7 Completeness

The translation is not complete, deliberately. Beyond the classical semantics (Remark 6.14), first-order provability from $\Delta(\mathbb{E}, G)$ lacks: induction (only case analysis is emitted; goals needing induction are the reconstruction back-end’s or the user’s job), conversion under binders for lifted symbols (two extensionally equal lifted functions get unrelated symbols — the ξ -rule failure discussed in [Cza18, Example 51]), and anything involving non-shallow propositions. A precise completeness theorem for a quantifier fragment (e.g. universal equational goals over first-order datatypes, where Proposition 6.12 makes $\mathfrak{M}$ standard) would be valuable; we expect the per-constructor equations to be complete for ground equational consequences of the definitions, by Remark 4.5 and standard rewriting arguments [BN98], but leave the statement to future work.

10.8 Proof-theoretic soundness and implementation

Two further directions. First, a proof-theoretic soundness theorem in the style of [Cza18] — transforming a first-order derivation from $\Delta(\mathbb{E}, G)$ into a CIC^- proof term, presumably modulo excluded middle and proof irrelevance — would sharpen Theorem 6.9 into a reconstruction algorithm; the substitution-stability machinery needed for lifted symbols is exactly the hard part there, and our semantic treatment sidesteps rather than solves it. Second, on the implementation side, the axiom families of Definition 4.10 map one-to-one onto sites in the existing CoqHammer translation (case lifting, guard generation, definition axioms), so the architecture can be adopted incrementally — per-constructor equations first, then singleton erasure and refinement expansion, then the full specification extraction — with each stage independently measurable against the current translation on the standard evaluation harness.

References

- [Acz77] Peter Aczel. An introduction to inductive definitions. In Jon Barwise, editor, *Handbook of Mathematical Logic*, pages 739–782. North-Holland, 1977.
- [BBPS16] Jasmin Christian Blanchette, Sascha Böhme, Andrei Popescu, and Nicholas Smallbone. Encoding monomorphic and polymorphic types. *Logical Methods in Computer Science*, 12(4), 2016.

- [BCD⁺11] Clark Barrett, Christopher L. Conway, Morgan Deters, Liana Hadarean, Dejan Jovanović, Tim King, Andrew Reynolds, and Cesare Tinelli. CVC4. In *Computer Aided Verification (CAV 2011)*, volume 6806 of *LNCS*, pages 171–177. Springer, 2011.
- [BFG⁺04] Gilles Barthe, Maria João Frade, Eduardo Giménez, Luís Pinto, and Tarmo Uustalu. Type-based termination of recursive definitions. *Mathematical Structures in Computer Science*, 14(1):97–141, 2004.
- [BKPU16] Jasmin Christian Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1):101–148, 2016.
- [BN98] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [CK16] Łukasz Czajka and Cezary Kaliszyk. Goal translation for a hammer for Coq (extended abstract). In *Proceedings First International Workshop on Hammers for Type Theories (HaTT 2016)*, volume 210 of *EPTCS*, pages 13–20, 2016.
- [CK18] Łukasz Czajka and Cezary Kaliszyk. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61(1–4):423–453, 2018.
- [CK26] Łukasz Czajka and Cezary Kaliszyk. CoqHammer: Automated reasoning for Rocq (Coq). <https://coqhammer.github.io/>, 2026.
- [Cza18] Łukasz Czajka. A shallow embedding of pure type systems into first-order logic. In *22nd International Conference on Types for Proofs and Programs (TYPES 2016)*, volume 97 of *LIPIcs*, pages 9:1–9:39. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.
- [dMB08] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008.
- [GCST19] Gaëtan Gilbert, Jesper Cockx, Matthieu Sozeau, and Nicolas Tabareau. Definitional proof-irrelevance without K. *Proceedings of the ACM on Programming Languages*, 3(POPL):3:1–3:28, 2019.

- [Gim95] Eduardo Giménez. Codifying guarded definitions with recursive schemes. In *Types for Proofs and Programs (TYPES 1994)*, volume 996 of *LNCS*, pages 39–59. Springer, 1995.
- [Kle45] Stephen Cole Kleene. On the interpretation of intuitionistic number theory. *Journal of Symbolic Logic*, 10(4):109–124, 1945.
- [Klo80] Jan Willem Klop. *Combinatory Reduction Systems*. PhD thesis, Utrecht University, 1980. Mathematical Centre Tracts 127.
- [Kra10] Alexander Krauss. Partial and nested recursive function definitions in higher-order logic. *Journal of Automated Reasoning*, 44(4):303–336, 2010.
- [KV13] Laura Kovács and Andrei Voronkov. First-order theorem proving and Vampire. In *Computer Aided Verification (CAV 2013)*, volume 8044 of *LNCS*, pages 1–35. Springer, 2013.
- [Let03] Pierre Letouzey. A new extraction for Coq. In *Types for Proofs and Programs (TYPES 2002)*, volume 2646 of *LNCS*, pages 200–219. Springer, 2003.
- [Let04] Pierre Letouzey. *Programmation fonctionnelle certifiée – L’extraction de programmes dans l’assistant Coq*. PhD thesis, Université Paris-Sud, 2004. HAL tel-00150912.
- [Let08] Pierre Letouzey. Extraction in Coq: An overview. In *Logic and Theory of Algorithms (CiE 2008)*, volume 5028 of *LNCS*, pages 359–369. Springer, 2008.
- [MP08] Jia Meng and Lawrence C. Paulson. Translating higher-order clauses to first-order clauses. *Journal of Automated Reasoning*, 40(1):35–60, 2008.
- [MW03] Alexandre Miquel and Benjamin Werner. The not so simple proof-irrelevant model of CC. In *Types for Proofs and Programs (TYPES 2002)*, volume 2646 of *LNCS*, pages 240–258. Springer, 2003.
- [PB12] Lawrence C. Paulson and Jasmin Christian Blanchette. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics (IWIL 2010)*, volume 2 of *EPiC Series in Computing*, pages 1–11, 2012.
- [PM89] Christine Paulin-Mohring. Extracting F_ω ’s programs from proofs in the calculus of constructions. In *Proceedings of the 16th ACM*

SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '89), pages 89–104. ACM, 1989.

- [QCBA25] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. Lean-auto: An interface between Lean 4 and automated theorem provers. In *Computer Aided Verification (CAV 2025)*. Springer, 2025. arXiv:2505.14929.
- [SBF⁺20] Matthieu Sozeau, Simon Boulier, Yannick Forster, Nicolas Tabareau, and Théo Winterhalter. Coq Coq correct! Verification of type checking and erasure for Coq, in Coq. *Proceedings of the ACM on Programming Languages*, 4(POPL):8:1–8:28, 2020.
- [Sch02] Stephan Schulz. E – a brainiac theorem prover. *AI Communications*, 15(2–3):111–126, 2002.
- [Sho67] Joseph R. Shoenfield. *Mathematical Logic*. Addison-Wesley, 1967.
- [SM19] Matthieu Sozeau and Cyprien Mangin. Equations reloaded: High-level dependently-typed functional programming and proving in Coq. *Proceedings of the ACM on Programming Languages*, 3(ICFP):86:1–86:29, 2019.
- [ŠR26] Artjoms Šinkarovs and Michael Rawson. When Agda met Vampire. arXiv:2602.18844, 2026.
- [Tak95] Masako Takahashi. Parallel reductions in λ -calculus. *Information and Computation*, 118(1):120–127, 1995.
- [The26] The Rocq Development Team. The Rocq prover reference manual. <https://rocq-prover.org/doc/>, 2026.
- [TS98] Tanel Tammet and Jan M. Smith. Optimized encodings of fragments of type theory in first-order logic. *Journal of Logic and Computation*, 8(6):713–744, 1998.
- [TvD88] Anne S. Troelstra and Dirk van Dalen. *Constructivism in Mathematics: An Introduction*. North-Holland, 1988.
- [Wer97] Benjamin Werner. Sets in types, types in sets. In *Theoretical Aspects of Computer Software (TACS 1997)*, volume 1281 of *LNCS*, pages 530–546. Springer, 1997.
- [Wie07] Freek Wiedijk. Mizar’s soft type system. In *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, volume 4732 of *LNCS*, pages 383–399. Springer, 2007.

A Confluence of $\lambda_{\square}$

We prove Theorem 3.3 by the Tait–Martin-Löf method in Takahashi’s formulation [Tak95], extended to the ι , μ and δ rules. Recall that the δ -rule unfolds a constant to the erasure of its body, $c \rightarrow_{\square} \mathcal{E}(t_c)$, and that bodies mention only earlier constants; no property of this ordering is needed below beyond the fact that each constant has a *fixed* unfolding.

Definition A.1 (Parallel reduction). The relation $\Rightarrow$ on $\lambda_{\square}$ terms is defined inductively:

1. $x \Rightarrow x$, $\square \Rightarrow \square$, $c \Rightarrow c$;
2. $c \Rightarrow \mathcal{E}(t_c)$ for a defined constant c ;
3. $\lambda x.e \Rightarrow \lambda x.e'$ if $e \Rightarrow e'$;
4. $e_1 e_2 \Rightarrow e'_1 e'_2$ if $e_i \Rightarrow e'_i$;
5. $c(\vec{e}) \Rightarrow c(\vec{e}')$ if $e_i \Rightarrow e'_i$ componentwise;
6. $\text{case}(e; \vec{b}r) \Rightarrow \text{case}(e'; \vec{b}r')$ if $e \Rightarrow e'$ and branch bodies reduce componentwise;
7. $\text{fix } f.e \Rightarrow \text{fix } f.e'$ if $e \Rightarrow e'$;
8. $(\lambda x.e) a \Rightarrow e'[a'/x]$ if $e \Rightarrow e'$ and $a \Rightarrow a'$;
9. $\text{case}(c_i(\vec{e}); \{c_j(\vec{x}_j) \Rightarrow b_j\}_j) \Rightarrow b'_i[\vec{e}'/\vec{x}_i]$ if $e_l \Rightarrow e'_l$ and $b_i \Rightarrow b'_i$;
10. $\text{fix } f.e \Rightarrow e'[\text{fix } f.e'/f]$ if $e \Rightarrow e'$.

Lemma A.2. (i) $e \Rightarrow e$ for all e . (ii) $\rightarrow_{\square} \subseteq \Rightarrow \subseteq \rightarrow_{\square}^*$. (iii) If $e \Rightarrow e'$ and $u \Rightarrow u'$ then $e[u/x] \Rightarrow e'[u'/x]$.

Proof. (i) Induction on e using clauses (1) and (3)–(7). (ii) For the first inclusion, a one-step reduction is a root rule applied inside a context: the corresponding root clause (2), (8), (9) or (10) with reflexive subderivations, propagated by the congruence clauses. For the second, induction on $e \Rightarrow e'$: each clause is realised by finitely many $\rightarrow_{\square}$ -steps (congruence clauses by the induction hypothesis; the redex clauses by reducing inside first, then one root step). (iii) Induction on the derivation of $e \Rightarrow e'$. The variable clause $x \Rightarrow x$ becomes $u \Rightarrow u'$; clause (2) is unaffected because $\mathcal{E}(t_c)$ is closed; the congruence clauses follow from the induction hypothesis; for the redex clauses one uses the standard substitution identities, e.g. $(e'[a'/y])[u'/x] = (e'[u'/x])[a'[u'/x]/y]$ for $y \notin \text{FV}(u') \cup \{x\}$. $\square$

Definition A.3 (Complete development). The complete development e^* is defined by recursion on e :

$$\begin{aligned}
x^* &= x, & \square^* &= \square, & c^* &= \mathcal{E}(t_c), \\
(\lambda x.e)^* &= \lambda x.e^*, & \mathbf{c}(\vec{e})^* &= \mathbf{c}(\vec{e}^*), & (\text{fix } f.e)^* &= e^*[\text{fix } f.e^*/f], \\
((\lambda x.e) a)^* &= e^*[a^*/x], & (e_1 e_2)^* &= e_1^* e_2^* \quad (e_1 \text{ not a } \lambda), \\
\text{case}(\mathbf{c}_i(\vec{e}); \vec{b}r)^* &= b_i^*[\vec{e}^*/\vec{x}_i], \\
\text{case}(e; \vec{b}r)^* &= \text{case}(e^*; \vec{b}r^*) \quad (e \text{ not a constructor form}).
\end{aligned}$$

(For a constant, the development unfolds once and does not recurse into the unfolding; for a fix, the root μ -redex is contracted once, its residuals in the contractum being new redexes that are not developed.)

Lemma A.4 (Triangle property). *If $e \Rightarrow e'$ then $e' \Rightarrow e^*$.*

Proof. Induction on the derivation of $e \Rightarrow e'$, with a case analysis on the last clause used.

- Clause (1): for variables and $\square$ immediate; for $c \Rightarrow c$ we need $c \Rightarrow c^* = \mathcal{E}(t_c)$, which is clause (2).
- Clause (2): $e' = \mathcal{E}(t_c) = c^*$, and $e' \Rightarrow e'$ by Lemma A.2(i).
- Clause (3), (5), (7)-congruence, and clause (6) with non-constructor scrutinee whose reduct is still handled uniformly: by the induction hypothesis componentwise. Two subtleties: for $\text{fix } f.e \Rightarrow \text{fix } f.e'$ (clause (7)) the target is $(\text{fix } f.e)^* = e^*[\text{fix } f.e^*/f]$, reached from $\text{fix } f.e'$ by clause (10) with $e' \Rightarrow e^*$ (induction hypothesis). For $\text{case}(e; \vec{b}r)$ with e not a constructor form, the scrutinee's reduct e'' may or may not be a constructor form, but the development $\text{case}(e^*; \vec{b}r^*)$ is reached by clause (6) either way.
- Clause (4) with $e_1 = \lambda x.b$: then $e_1 \Rightarrow e'_1$ is by clause (3), $e'_1 = \lambda x.b'$, and $e^* = b^*[a^*/x]$; from $e' = (\lambda x.b') a'$ apply clause (8) with $b' \Rightarrow b^*$, $a' \Rightarrow a^*$ (induction hypotheses).
- Clause (4) with e_1 not a λ : $e^* = e_1^* e_2^*$; apply clause (4) with the induction hypotheses. (If e'_1 happens to be a λ or a constant unfolding, no root contraction is required of us.)
- Clause (8): $e = (\lambda x.b) a$, $e' = b'[a'/x]$; by induction hypotheses $b' \Rightarrow b^*$ and $a' \Rightarrow a^*$, so Lemma A.2(iii) gives $e' \Rightarrow b^*[a^*/x] = e^*$.
- Clause (9): as clause (8), with the componentwise induction hypotheses for the constructor arguments and the branch, and Lemma A.2(iii) extended to simultaneous substitutions. Note the scrutinee $\mathbf{c}_i(\vec{e})$ can only have been reduced componentwise (clause (5)), so the case analysis in Definition A.3 is stable.

- Clause (10): $e = \text{fix } f.b$, $e' = b'[\text{fix } f.b'/f]$; by the induction hypothesis $b' \Rightarrow b^*$, whence $\text{fix } f.b' \Rightarrow \text{fix } f.b^*$ by clause (7), and Lemma A.2(iii) gives $e' \Rightarrow b^*[\text{fix } f.b^*/f] = e^*$.

□

Proof of Theorem 3.3. By Lemma A.4, $\Rightarrow$ has the diamond property: if $e \Rightarrow e_1$ and $e \Rightarrow e_2$ then $e_1 \Rightarrow e^*$ and $e_2 \Rightarrow e^*$. A relation with the diamond property has confluent reflexive-transitive closure (tiling), and by Lemma A.2(ii) the closure of $\Rightarrow$ is $\rightarrow_{\square}^*$. The characterisation of $=_{\square}$ by common reducts follows by induction on the conversion zig-zag (Church–Rosser). □

$$\begin{array}{c}
\frac{p : \varphi \in \Gamma}{\Gamma \vdash p : \varphi} \quad \frac{(c := \pi : \varphi) \in \mathbb{E}}{\Gamma \vdash c : \varphi} \quad \frac{\Gamma, p : \varphi \vdash \pi : \psi}{\Gamma \vdash \lambda p : \varphi. \pi : \varphi \rightarrow \psi} \\
\frac{\Gamma \vdash \pi : \varphi \rightarrow \psi \quad \Gamma \vdash \pi' : \varphi}{\Gamma \vdash \pi \pi' : \psi} \quad \frac{\Gamma, x : \sigma \vdash \pi : \varphi}{\Gamma \vdash \lambda x : \sigma. \pi : \forall x : \sigma. \varphi} \\
\frac{\Gamma \vdash \pi : \forall x : \sigma. \varphi \quad \Gamma \vdash a : \sigma}{\Gamma \vdash \pi a : \varphi[a/x]} \quad \frac{\Gamma, q : K \vdash \pi : \varphi}{\Gamma \vdash \Lambda q : K. \pi : \forall q : K. \varphi} \\
\frac{\Gamma \vdash \pi : \forall q : K. \varphi \quad \Gamma \vdash P : K}{\Gamma \vdash \pi P : \varphi[P/q]} \quad \frac{\Gamma \vdash \pi_1 : \varphi_1 \quad \Gamma \vdash \pi_2 : \varphi_2}{\Gamma \vdash \langle \pi_1, \pi_2 \rangle : \varphi_1 \wedge \varphi_2} \\
\frac{\Gamma \vdash \pi : \varphi_1 \wedge \varphi_2}{\Gamma \vdash \pi.i : \varphi_i} \quad \frac{\Gamma \vdash \pi : \varphi_i}{\Gamma \vdash \iota_i(\pi) : \varphi_1 \vee \varphi_2} \\
\frac{\Gamma \vdash \pi : \varphi_1 \vee \varphi_2 \quad \Gamma, p_i : \varphi_i \vdash \pi_i : \psi \ (i = 1, 2)}{\Gamma \vdash \text{vcase}(\pi; p_1.\pi_1; p_2.\pi_2) : \psi} \\
\frac{\Gamma \vdash a : \sigma \quad \Gamma \vdash \pi : \varphi[a/x]}{\Gamma \vdash \langle a, \pi \rangle : \exists x : \sigma. \varphi} \quad \frac{}{\Gamma \vdash \text{tt} : \top} \quad \frac{\Gamma \vdash a : \sigma}{\Gamma \vdash \text{refl}_\sigma(a) : a =_\sigma a} \\
\frac{\Gamma \vdash \pi : \exists x : \sigma. \varphi \quad \Gamma, x : \sigma, p : \varphi \vdash \pi' : \psi \quad x, p \notin \text{FV}(\psi)}{\Gamma \vdash \text{ecase}(\pi; x p. \pi') : \psi} \quad \frac{\Gamma \vdash \pi : \perp \quad \Gamma \vdash \varphi \text{ prop}}{\Gamma \vdash \text{efq}(\pi; \varphi) : \varphi} \\
\frac{\Gamma \vdash t : \{x : \sigma \mid \varphi\}}{\Gamma \vdash \text{valid}(t) : \varphi[\text{val}(t)/x]} \quad \frac{\Gamma \vdash \pi : \varphi \quad \varphi \simeq \varphi' \quad \Gamma \vdash \varphi' \text{ prop}}{\Gamma \vdash \pi : \varphi'} \\
\frac{\Gamma \vdash \pi : a =_\sigma b \quad \Gamma, z : \sigma \vdash \psi \text{ prop} \quad \Gamma \vdash \pi' : \psi[a/z]}{\Gamma \vdash \text{rew}(\pi; z. \psi; \pi') : \psi[b/z]} \\
\frac{\mathbf{k} : \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u}) \in \mathbb{E} \quad \Gamma \vdash \vec{a} : \vec{\tau} \quad \Gamma \vdash \pi_i : \varphi_i[\vec{a}/\vec{x}] \text{ (all } i\text{)}}{\Gamma \vdash \mathbf{k}(\vec{a}; \vec{\pi}) : J(\vec{u}[\vec{a}/\vec{x}])} \\
\frac{\Gamma \vdash \pi : J(\vec{a}) \quad \Gamma, \vec{z} : \vec{\sigma} \vdash \psi \text{ prop} \quad \Gamma \vdash \pi_j : \Xi_j(\vec{z}. \psi) \text{ (all clauses } j\text{)}}{\Gamma \vdash \text{ind}_J(\vec{z}. \psi; \vec{\pi}; \pi) : \psi[\vec{a}/\vec{z}]} \\
\frac{\Gamma \vdash a : I \quad \Gamma, z : I \vdash \psi \text{ prop} \quad \Gamma \vdash \pi_i : \Theta_i(z. \psi) \text{ (all constrs } i\text{)}}{\Gamma \vdash \text{ind}_I(z. \psi; \vec{\pi}; a) : \psi[a/z]}
\end{array}$$

Figure 3: Typing of proofs. For a predicate clause $\mathbf{k}_j : \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow J(\vec{u})$, the *clause obligation* is $\Xi_j(\vec{z}. \psi) = \forall \vec{x} : \vec{\tau}. \varphi_1 \rightarrow \dots \rightarrow \varphi_\ell \rightarrow \psi[\vec{v}_1/\vec{z}] \rightarrow \dots \rightarrow \psi[\vec{v}_r/\vec{z}] \rightarrow \psi[\vec{u}/\vec{z}]$, where $J(\vec{v}_1), \dots, J(\vec{v}_r)$ are the recursive premises among the φ_i (an induction hypothesis for each). For a datatype constructor $\mathbf{c}_i : (\vec{\sigma}_i)$ of I , the clause obligation is $\Theta_i(z. \psi) = \forall \vec{y} : \vec{\sigma}_i. \psi[y_{r_1}/z] \rightarrow \dots \rightarrow \psi[y_{r_s}/z] \rightarrow \psi[\mathbf{c}_i(\vec{y})/z]$, where $y_{r_1}, \dots, y_{r_s}$ are the recursive arguments.